

**DIRE DAWA UNIVERSITY
DIRE DAWA INSTITUTE OF TECHNOLOGY
SCHOOL OF COMPUTING
DEPARTMENT OF SOFTWARE ENGINEERING**

PROJECT PROPOSAL

ON

**“Design and Implementation of Temariware:
A Telegram Mini App for University Student
Job Matching in Ethiopia”**

A Proposal Submitted to the Dire Dawa University Institute of Technology School of Computing Department of Software Engineering of Dire Dawa University in Partial Fulfilment of the Requirements for the Degree of BSc in Software Engineering

BY:

Nebiyu Tsegaye

ID: 1501332

ADVISOR:

[Advisor Name (Title)]

[Month, Year]

Dire Dawa, Ethiopia

CERTIFICATE

This is to certify that the project proposal entitled “Design and Implementation of Temariware: A Telegram Mini App for University Student Job Matching in Ethiopia” has been carried out by Nebiyu Tsegaye (ID: 1501332) under my supervision and guidance as a partial fulfilment of the requirements for the Degree of BSc in Software Engineering at the Department of Software Engineering, School of Computing, Dire Dawa Institute of Technology, Dire Dawa University.

To the best of my knowledge, the work presented in this proposal is original, has not been submitted elsewhere for the award of any other degree or diploma, and the contributions of others have been appropriately acknowledged through citations and references.

[Advisor Name (Title)]

Senior Project Advisor

Department of Information Technology

Dire Dawa University

Date: _____

ACKNOWLEDGMENT

First and foremost, we would like to express our deepest gratitude to our project advisor, [Advisor Name (Title)], for the unwavering guidance, constructive feedback, and continuous encouragement throughout the course of this project. His/her insights into software engineering principles and Telegram Mini App development were instrumental in shaping the direction of this work, and his/her patience in reviewing multiple drafts of this proposal significantly improved its quality.

We also extend our sincere thanks to the academic staff of the Department of Information Technology, School of Computing, Dire Dawa Institute of Technology, for the

foundational knowledge they imparted during our studies. The courses on database management systems, software engineering, web application development, and human-computer interaction provided the theoretical and practical grounding necessary to undertake a project of this scope. We are particularly grateful to the department head and the project coordinator for organising the industrial project programme and for making the project guideline document available, which served as the structural backbone of this proposal.

Our appreciation also goes to the broader Ethiopian Telegram developer community, whose open-source examples of Telegram Bot API integrations and Mini App implementations were valuable references during our research. Finally, we acknowledge the love, patience, and financial support of our families throughout our university education; without their encouragement, completing this project would not have been possible. Any remaining errors or omissions in this document are solely our responsibility.

ABSTRACT

Youth unemployment is one of the most pressing socio-economic challenges facing Ethiopia today, with university graduates often struggling to find employment that matches their skills despite a steady demand for tutoring, freelance, and part-time labour in cities and towns across the country. Although job opportunities are regularly broadcast through informal Telegram channels such as Afriwork and Yegna Tutors Harrar, the lack of a structured platform means that opportunities are easily missed, applicants cannot be reliably verified, and employers have no way to manage applications or collect fees in a transparent manner. This project, Temariware, proposes the design and implementation of a Telegram Mini App combined with a Telegram Bot that addresses these gaps by providing a single, multilingual platform where Ethiopian university students can browse verified jobs, apply with their academic profile, and pay a confirmation fee through a screenshot-based verification flow, while employers can post jobs, review applicant documents (National ID, University ID, and last semester grade report), accept candidates, and have the platform automatically close positions once hiring targets are met.

The system is built using a modern full-stack architecture: a Next.js 16 web application with React 19 and TypeScript serves as the Mini App frontend, a Prisma-managed PostgreSQL database hosted on Neon stores all user, job, application, and payment records, and the application is deployed on Vercel's free tier. Authentication is performed through

Telegram's WebApp initData HMAC-SHA256 validation, ensuring that every user is a real Telegram account holder. The bot, accessible as @temariwarebot, exposes slash commands (/start, /latest, /search, /subscribe, /postjob, /admin) and pushes instant job alerts to subscribers, while the @TEMARIWARE channel automatically receives richly-formatted job cards every time an administrator approves a new posting.

The proposed system is expected to reduce the friction of job discovery for students, increase trust between employers and applicants through mandatory document verification, and provide a sustainable revenue model through configurable per-job application fees. The complete software stack operates on free-tier services, making the platform accessible to students without any upfront cost. This proposal document presents the problem background, objectives, scope, methodology, system requirements, analysis, and preliminary design of Temariware in accordance with the Dire Dawa University industrial project guideline.

Keywords: Telegram Mini App, Job Matching, University Students, Ethiopia, Next.js, Prisma, Neon Postgres, Telegram Bot API, Payment Verification, Document Verification.

TABLE OF CONTENTS

CERTIFICATE.....	4
ACKNOWLEDGMENT.....	6
ABSTRACT.....	7
TABLE OF CONTENTS.....	8
LIST OF FIGURES.....	8
LIST OF TABLES.....	10
ACRONYMS.....	11
CHAPTER ONE: INTRODUCTION.....	11
1.1 Introduction.....	11
1.2 Background (Overview) of the System.....	12
1.3 Statement of the Problem.....	12

1.4 General and Specific Objectives.....	13
1.4.1 General Objective.....	13
1.4.2 Specific Objectives.....	13
1.5 Scope of the Project.....	15
1.6 Limitation of the Project.....	15
1.7 Methodology and Approaches, Techniques and Tools.....	16
1.7.1 Methodology and Approaches.....	16
1.7.2 Techniques.....	16
1.7.3 Tools.....	17
1.8 Significance of the Project.....	18
1.9 Work Breakdown Structure and Feasibility Analysis.....	19
1.9.1 Work Breakdown.....	19
1.9.2 Feasibility Analysis.....	19
CHAPTER TWO: SYSTEM REQUIREMENT SPECIFICATION AND ANALYSIS.....	20
2.1 Introduction/Overview.....	20
2.2 Description of the Current System.....	21
2.3 Functional Requirements of the Proposed System.....	21
2.4 Non-Functional Requirements (Technical Requirements).....	21
2.5 Supplementary Specification.....	22
2.5.1 Business Rules.....	22
2.5.2 Constraint.....	24
2.5.3 Change Case.....	25
2.5.4 Requirement Elicitation Models.....	26
2.5.4.1 Essential Use Case Modeling.....	26
2.5.4.2 CRC Modeling.....	28

2.5.4.3 Essential User Interface (UI) Prototype.....	29
2.5.4.4 User Interface (UI) Flow Diagram.....	29
2.6 The Proposed System Models (in OO Approach).....	30
2.6.1 System Use Case Model.....	30
2.6.2 Conceptual Class Model.....	31
2.6.3 Sequence Diagram.....	31
2.6.4 Activity Diagram.....	32
2.6.5 User Interface Prototype.....	33
CHAPTER THREE: DESIGN SPECIFICATION.....	34
3.1 Introduction/Overview.....	34
3.2 Software Architecture.....	34
3.2.1 Architectural Styles.....	35
3.2.2 Describing an Architecture using UML.....	35
3.2.2.1 Class Type Architecture.....	35
3.2.2.2 Component Diagram.....	36
3.2.2.3 Deployment Diagram.....	36
3.2.2.4 Persistent Diagram.....	37
3.3 Detailed Design of the System.....	37
3.3.1 Design Level Class Diagram.....	38
3.3.2 Collaboration Diagram.....	38
3.3.3 State Chart Diagram.....	39
3.3.4 User Interface Diagram.....	40
REFERENCES.....	41
APPENDICES.....	45
Appendix A: Complete CRC Card Set.....	45
Appendix B: User Interface Prototype Screenshots.....	45

Appendix C: Database Schema (Prisma).....	47
Appendix D: Telegram Bot Commands Reference.....	48
Appendix E: Deployment Guide Summary.....	49

[Right-click the table above and choose “Update Field” to refresh page numbers after editing.]

LIST OF FIGURES

[Figure 2.1: Essential Use Case Diagram of Temariware	Error!
Bookmark not defined.]	
[Figure 2.2: Conceptual Class Model	Error! Bookmark
not defined.]	
[Figure 2.3: Sequence Diagram — Apply to Job	Error! Bookmark
not defined.]	
[Figure 2.4: Activity Diagram — Job Posting & Approval Flow	Error! Bookmark
not defined.]	
[Figure 2.5: Essential User Interface Prototype — Home View	Error! Bookmark
not defined.]	
[Figure 3.1: Component Diagram of Temariware	Error! Bookmark
not defined.]	
[Figure 3.2: Deployment Diagram	Error! Bookmark
not defined.]	
[Figure 3.3: Design-Level Class Diagram	Error! Bookmark
not defined.]	
[Figure 3.4: State Chart Diagram — Application Lifecycle	Error! Bookmark
not defined.]	
[Figure 3.5: User Interface Diagram — Job Detail View	Error! Bookmark
not defined.]	

LIST OF TABLES

[Table 1.1: Project Work Breakdown Structure	Error! Bookmark not defined.]
[Table 1.2: Feasibility Analysis Summary	Error! Bookmark not defined.]
[Table 2.1: Functional Requirements	Error! Bookmark not defined.]
[Table 2.2: Non-Functional Requirements	Error! Bookmark not defined.]
[Table 2.3: CRC Card — User Class	Error! Bookmark not defined.]
[Table 3.1: Software Architecture Styles Considered	Error! Bookmark not defined.]
[Table 3.2: Persistent Data Model Summary	Error! Bookmark not defined.]

ACRONYMS

API	Application Programming Interface
BSc	Bachelor of Science
CBE	Commercial Bank of Ethiopia
CRC	Class-Responsibility-Collaboration
CV	Curriculum Vitae
DBMS	Database Management System
ETB	Ethiopian Birr
HMAC	Hash-based Message Authentication Code
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
ID	Identification
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
OOP	Object-Oriented Programming
ORM	Object-Relational Mapping
PDF	Portable Document Format
PNG	Portable Network Graphics
REST	Representational State Transfer
SDK	Software Development Kit
SHA	Secure Hash Algorithm
SQL	Structured Query Language
TCP	Transmission Control Protocol
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
VPS	Virtual Private Server
WAF	Web Application Firewall
XML	Extensible Markup Language

CHAPTER ONE: INTRODUCTION

1.1 Introduction

The rapid expansion of mobile internet access and the widespread adoption of instant messaging platforms in Ethiopia have transformed how young people search for employment opportunities. Telegram, in particular, has emerged as the de-facto communication backbone for university students, freelancers, and small businesses across the country, with millions of daily active users exchanging text, files, and payments through the platform. Within this ecosystem, a number of informal job-broadcasting channels have sprung up — channels such as Afriwork (Freelance Ethiopia) and Yegna Tutors Harrar regularly post tutoring gigs, part-time sales positions, freelance design work, and even full-time software development roles aimed at university students. These channels are popular because they are free to join, reach a large audience instantly, and feel native to the platforms students already use every day.

However, the existing channel-based model suffers from several structural limitations. Job posts are unstructured free text, making it difficult to filter by subject, location, or salary. There is no centralised repository of past postings, so opportunities disappear into the chat history within hours. Applicants have no way to verify the legitimacy of employers, and employers have no way to verify the identity, academic standing, or contact information of applicants before scheduling an interview. There is also no mechanism for collecting posting or application fees, which limits the sustainability of these channels as businesses and creates friction when employers genuinely want to filter serious applicants from casual browsers. Temariware — the system proposed in this document — is designed to solve all of these problems by combining the reach of Telegram with the structure of a traditional job-matching platform.

This chapter introduces the project by laying out the background of the existing system, articulating the problem statement, defining the general and specific objectives, scoping the work, listing its limitations, describing the methodology and tools used, explaining its significance, and finally presenting the work breakdown structure and feasibility analysis. The remaining chapters cover system requirement specification and analysis (Chapter Two) and design specification (Chapter Three).

1.2 Background (Overview) of the System

The Ethiopian higher education sector has expanded dramatically over the past two decades, with dozens of public universities now enrolling hundreds of thousands of undergraduate students every year. Despite this growth, formal part-time employment opportunities for students remain scarce and poorly advertised. Most universities do not maintain an internal job board, and the few commercial job portals that exist in Ethiopia — such as EthioJobs and Ezega Jobs — focus almost exclusively on full-time professional roles aimed at graduates rather than part-time tutoring, freelance, or internship roles suited to enrolled students. This gap has driven students and employers alike to informal channels on Telegram, where posting is free and the audience is highly targeted.

Telegram's bot platform, first released in 2015, has matured into a capable development environment. The introduction of the Telegram Mini App framework (formerly Web Apps) in 2022 allowed bots to launch full-screen web applications inside the Telegram client, with the user's identity automatically passed to the application through a cryptographically signed initData string. This means that a well-designed Mini App can offer a rich user interface — complete with forms, image uploads, payments, and navigation — while inheriting the user's Telegram identity, contacts, and chat context. Combined with the Telegram Bot API's webhook mechanism, this enables real-time bidirectional communication between a backend server and the end user, all without the user ever leaving the Telegram app.

Several existing bots in the Ethiopian market already use parts of this stack. Afriwork's bot (@freelanceethbot) broadcasts job posts and accepts applications via forwarded messages. Yegna Tutors Harrar (@Yegnatutorharrar) operates similarly but focuses exclusively on tutoring roles. Neither bot, however, offers a structured Mini App for browsing or applying, neither verifies user identity through documents, and neither supports integrated payments. Temariware's contribution is to combine all of these capabilities into a single, free-to-use platform that is specifically tailored to the needs of Ethiopian university students and the employers who want to hire them.

1.3 Statement of the Problem

Despite the popularity of informal Telegram job channels, the current model fails both students and employers in several measurable ways. First, job posts are unstructured: a typical post from Yegna Tutors might list subjects, schedule, location, and salary as free-

flowing text, making it impossible for a student to filter by subject or location, and easy to miss relevant opportunities when scrolling through a long chat history. Second, there is no persistent record of past postings, so once a job disappears from the recent feed it is effectively lost forever. Third, neither employers nor students are verified, which creates an environment where scams, no-shows, and disputes are common and trust is low.

Fourth, the application process is highly manual: a student who is interested in a job must forward their CV and contact details to the employer's personal Telegram account, after which the employer must manually review each application in their private chat. There is no way to track which jobs a student has applied to, no way to compare applicants side by side, and no way to enforce a structured workflow such as “accept → pay confirmation fee → hired.” Fifth, the existing channels have no monetisation model that would allow them to grow into sustainable businesses; without a payment mechanism, channel operators cannot charge for premium features, and employers cannot filter out casual applicants by requiring a small commitment fee.

Temariware addresses all of these problems by providing a structured, verified, payment-enabled job-matching platform built directly into Telegram. The system allows employers to post jobs with rich, filterable metadata; requires students to upload National ID, University ID, and last semester grade report before they can apply; supports a per-job application fee paid via Telebirr or CBE with screenshot-based verification; and automatically closes positions once the specified number of hires has been reached. By solving these problems in a single integrated platform, Temariware aims to significantly reduce the friction of part-time employment for Ethiopian university students while creating a sustainable, trustworthy marketplace for employers.

1.4 General and Specific Objectives

1.4.1 General Objective

The general objective of this project is to design, develop, and deploy a Telegram Mini App and Bot called Temariware that connects Ethiopian university students with verified tutoring, freelance, part-time, and full-time job opportunities, while providing integrated payment verification, mandatory document checks, and automated hiring workflow management.

1.4.2 Specific Objectives

To achieve the general objective, the following specific objectives have been defined:

- i. To analyse the limitations of existing informal Telegram job channels and elicit detailed functional and non-functional requirements from stakeholders (students, employers, and administrators).
- ii. To design a multi-section Mini App user interface — covering job feed, job detail, post-job form, applications panel, profile, and admin panel — that is usable on small mobile screens and supports English, Amharic, and Afaan Oromoo.
- iii. To design and implement a PostgreSQL database schema (managed via Prisma ORM) that captures users, jobs, applications, payments, subscriptions, and verification requests with appropriate relationships and constraints.
- iv. To implement a Telegram Bot webhook that handles slash commands (/start, /latest, /search, /subscribe, /postjob, /admin, /help), pushes instant job alerts to subscribers, and broadcasts approved jobs to the @TEMARIWARE channel.
- v. To implement Telegram WebApp initData HMAC-SHA256 validation so that every API request is cryptographically tied to a real Telegram user identity.
- vi. To implement a screenshot-based payment verification flow that allows employers to pay a configurable job post fee and students to pay a per-job application fee upon acceptance, with all payments reviewed and approved by an administrator.
- vii. To implement mandatory student document upload (National ID, University ID, last semester grade report) with image compression, and expose these documents to employers during applicant review.
- viii. To implement automatic job closure when the number of hired applicants reaches the positionsAvailable threshold, with appropriate notifications to all stakeholders.
- ix. To deploy the complete system on free-tier infrastructure (Vercel + Neon Postgres + Telegram Bot API) and verify end-to-end functionality through browser-based and bot-based testing.
- x. To document the system comprehensively in accordance with the Dire Dawa University industrial project guideline, including this proposal, the requirement specification, the design specification, and references.

1.5 Scope of the Project

Temariware is scoped as a v1 production system that delivers end-to-end job posting, application, payment, and hiring functionality for the Ethiopian university student market. The system supports three primary user roles: students (who browse, apply, and pay), employers (who post, review, and accept), and administrators (who approve, verify

payments, and manage settings). The platform supports three languages (English, Amharic, Afaan Oromoo) and is designed to work on any device that runs the Telegram mobile, desktop, or web client.

Within scope are: the Next.js Mini App with six primary views (Home, Job Detail, Post Job, Applications, Profile, Admin); the Telegram bot with nine slash commands and webhook handler; a PostgreSQL database with seven primary entities (User, Job, Application, Subscription, VerificationRequest, Setting, Payment); an HMAC-based authentication layer; a screenshot-based payment verification flow with admin approval; mandatory document upload with client-side image compression; per-job configurable application fees; automatic job closure upon hiring target completion; channel broadcast to @TEMARIWARE; and per-subscriber push alerts.

Explicitly out of scope for v1 are: integration with real Chapa or Telebirr API for automated payment verification (only screenshot-based verification is implemented; a Chapa integration stub is included for v2); automated scraping and import of jobs from existing Telegram channels (manual posting only); in-app CV file storage (CV is referenced via external Google Drive or Dropbox links); email or SMS notifications (Telegram-only); employer dashboards with multiple job management (single-job-at-a-time applicant review); saved/bookmarked jobs; and reporting or analytics beyond basic admin stats. These features are documented as future work in the conclusion.

1.6 Limitation of the Project

Several limitations of the proposed system should be acknowledged at the proposal stage. First, the payment verification flow relies on manual review of payment screenshots by an administrator. This introduces a latency of several hours between a student uploading a payment and being marked as hired, and it creates a single point of failure if the administrator is unavailable. A future version of the system would integrate with Chapa or Telebirr's API for instant automated verification, but this requires a registered business entity and a payment processor account that the project team does not currently possess.

Second, the system depends entirely on Telegram as the delivery platform. Users who do not have a Telegram account cannot use Temariware, and any outage or API change on Telegram's side would directly impact the system's availability. Third, the system stores payment screenshots and identity documents as base64-encoded strings inside the PostgreSQL database rather than in dedicated object storage such as Supabase Storage or AWS S3. This decision was made to keep the deployment simple and within free-tier limits,

but it imposes a practical limit of approximately one megabyte per image and may require migration to object storage as the user base grows.

Fourth, the platform's trust model is based on document upload and admin verification, but it does not perform automated authenticity checks on uploaded IDs. A determined bad actor could upload a forged National ID image, and the system would have no automated way to detect this. Mitigation relies on the administrator visually inspecting each document and on the legal deterrent of fraudulent identity use under Ethiopian law. Fifth, the system currently supports only the Ethiopian market and the three languages mentioned; supporting other East African countries or additional languages would require additional localisation work. Finally, the system does not currently include an in-app dispute resolution mechanism; disputes between students and employers must be handled through the existing Telegram chat between the two parties.

1.7 Methodology and Approaches, Techniques and Tools

1.7.1 Methodology and Approaches

The Temariware project follows the Object-Oriented Software Development approach as recommended by the Dire Dawa University industrial project guideline. Within this broad approach, the team adopts an iterative and incremental development process, with each iteration producing a working slice of functionality that can be demonstrated to the advisor and tested by potential users. The methodology is loosely based on the Agile Unified Process, combining lightweight upfront design (use case diagrams, conceptual class diagrams) with continuous integration and frequent deployment to a staging environment on Vercel.

The project lifecycle consists of five phases: (1) requirements elicitation and analysis, in which existing Telegram job channels are studied, stakeholder interviews are conducted, and use cases are documented; (2) system design, in which the database schema, API contracts, and user interface are designed; (3) implementation, in which the Next.js Mini App, Prisma schema, and Telegram bot webhook are coded; (4) testing, in which end-to-end flows are verified through browser automation and manual bot interaction; and (5) deployment, in which the system is published to Vercel, the Telegram webhook is registered, and the @TEMARIWARE channel is configured for broadcasting.

1.7.2 Techniques

Several specific software engineering techniques are employed. Object-oriented analysis is used to identify the primary entities (User, Job, Application, Payment, Subscription, VerificationRequest, Setting) and their relationships. Use case modelling is used to capture the interactions between actors (Student, Employer, Admin, Bot) and the system. CRC (Class-Responsibility-Collaboration) modelling is used during the conceptual design phase to assign responsibilities to each class. Sequence diagrams are used to document the multi-step flows such as “apply to job,” “accept applicant,” and “verify payment.” Activity diagrams capture the parallel and conditional flows such as the job approval and broadcast pipeline. Client-side image compression using the HTML Canvas API is used to keep uploaded document and payment screenshot sizes within database limits. Cryptographic HMAC-SHA256 validation is used to verify the authenticity of Telegram initData strings.

1.7.3 Tools

The following tools are used throughout the project:

- Next.js 16 with App Router — the primary web framework for the Mini App, providing server-rendered React 19 components, API routes, and TypeScript support.
- React 19 + TypeScript 5 — the frontend library and type system used to build the user interface components.
- Tailwind CSS 4 with shadcn/ui — the styling framework and component library that provides a consistent, Telegram-native visual design.
- Prisma ORM 6 — the object-relational mapper that manages database schema, migrations, and typed queries.
- PostgreSQL on Neon — the relational database hosting provider, selected for its generous free tier (500MB storage) and Frankfurt region proximity to Ethiopia.
- Vercel — the deployment platform that hosts the Next.js application on its free tier with automatic HTTPS and global CDN.
- Telegram Bot API — the official Telegram API used to receive updates via webhook, send messages, set the menu button, and configure bot commands.
- Bun — the JavaScript runtime and package manager used during local development for its speed and built-in TypeScript execution.
- Visual Studio Code — the integrated development environment used for code editing, debugging, and version control integration.

- Git and GitHub — the version control system and remote repository used to track changes and collaborate within the team.
- Postman — the API testing tool used during development to verify endpoint behaviour.
- Vercel CLI — the command-line interface used to deploy the application and manage environment variables.

1.8 Significance of the Project

The significance of Temariware can be understood from the perspectives of three primary stakeholder groups: students, employers, and the broader Ethiopian tech ecosystem. For students, the platform significantly reduces the friction of finding part-time employment while studying. Instead of monitoring dozens of informal Telegram channels and manually forwarding CVs to employers, students can subscribe to subject- or location-based alerts on a single bot, browse a structured job feed, and apply with a single tap using a pre-verified profile. The mandatory document upload (National ID, University ID, grade report) serves a dual purpose: it builds trust with employers who would otherwise have no way to verify a student's identity, and it gives the student an incentive to maintain a complete and accurate academic record.

For employers, the platform provides a structured applicant tracking workflow that simply does not exist in the current informal channel model. Employers can post jobs with rich metadata, set a per-job application fee that filters out casual browsers, review applicants' full profiles including documents in a single panel, accept or reject with one tap, and have the platform automatically close the position once the hiring target is met. The screenshot-based payment flow, while not as automated as a full Chapa integration, still provides a transparent record of every transaction and gives the administrator a clear audit trail. The @TEMARIWARE broadcast channel extends the reach of every posting to a public audience, including people who have not yet subscribed to the bot.

For the broader Ethiopian tech ecosystem, Temariware serves as a reference implementation of a production-grade Telegram Mini App built entirely on free-tier infrastructure. The complete source code, deployment guide, and architectural decisions documented in this proposal can serve as a learning resource for other student developers who wish to build similar platforms. The project also demonstrates that it is possible to launch a real, revenue-generating software product in Ethiopia without any upfront capital investment in servers or proprietary software licences, which has implications for youth entrepreneurship and digital economy policy. Finally, by formalising a previously informal

labour market, the platform creates data that can inform future research on youth employment patterns in Ethiopia.

1.9 Work Breakdown Structure and Feasibility Analysis

1.9.1 Work Breakdown

The project is divided into six major work packages, each with clearly defined deliverables, estimated effort, and responsible team members. The breakdown is summarised in Table 1.1 below and presented in detail in the project plan that accompanies this proposal.

Table 1.1: Project Work Breakdown Structure

ID	Work Package	Duration	Responsible	Deliverables
WP1	Requirements Elicitation & Analysis	2 weeks	All members	Requirements doc, use cases
WP2	System Design	2 weeks	Member 1, 2	Schema, API contracts, UI mockups
WP3	Database & Backend Implementation	3 weeks	Member 1, 3	Prisma schema, API routes, auth
WP4	Frontend Mini App Implementation	3 weeks	Member 2, 4	All views, navigation, i18n
WP5	Bot Webhook & Channel Integration	1 week	Member 3	Bot commands, broadcasts, alerts
WP6	Testing, Deployment, Documentation	1 week	All members	Deployed app, this document

The total estimated effort for the project is approximately sixteen person-weeks, distributed across the four team members over a calendar period of roughly twelve weeks. The requirements and design phases are front-loaded to ensure that the implementation phase begins with a clear specification, while testing and deployment are interspersed with implementation rather than left to the end. The advisor review milestones are aligned with the end of each phase to ensure continuous feedback.

1.9.2 Feasibility Analysis

A feasibility analysis was conducted along four traditional dimensions: technical, economic, operational, and schedule feasibility. The analysis is summarised in Table 1.2 below.

Table 1.2: Feasibility Analysis Summary

Dimension	Analysis	Verdict
Technical	All tools are free, open-source, and well-documented. Team has relevant coursework background.	Feasible
Economic	Zero monthly recurring cost on Vercel free tier + Neon free tier + Telegram Bot API. Only cost is team time.	Feasible
Operational	Aligns with existing Telegram usage habits. No user training required. Document verification is intuitive.	Feasible
Schedule	12-week timeline with 4 team members. Iterative development with continuous integration mitigates schedule risk.	Feasible (tight)

The analysis indicates that the project is feasible on all four dimensions. Technically, all required tools and platforms are freely available and well-documented, and the team has the necessary background in web development and databases from prior coursework. Economically, the total recurring cost is zero Ethiopian Birr per month on free-tier infrastructure, with the only cost being the team's time. Operationally, the system aligns with existing user behaviour (Telegram usage is already widespread) and requires no training for end users. From a schedule perspective, the twelve-week timeline is tight but achievable given the iterative development approach and the fact that the team has already begun implementation as part of the proposal phase. The main risk is the dependency on a single administrator (the project team) for payment verification, which will be mitigated in v2 through Chapa API integration.

CHAPTER TWO: SYSTEM REQUIREMENT SPECIFICATION AND ANALYSIS

2.1 Introduction/Overview

This chapter presents the requirement specification and analysis for the Temariware system. It is organised into two parts: Part One (System Requirement Specification) describes the current system, elicits the functional and non-functional requirements of the proposed system, and documents the supplementary specifications including business rules, constraints, and change cases. Part Two (System Analysis) presents the proposed system models in the Object-Oriented approach, including the system use case model, conceptual class model, sequence diagram, activity diagram, and user interface prototype. Together, these two parts form the bridge between the high-level objectives defined in Chapter One and the detailed design that will be presented in Chapter Three.

The requirements were elicited through three complementary techniques: (1) direct observation of existing informal Telegram job channels such as Afriwork (@freelanceethbot) and Yegna Tutors Harrar (@Yegnatutorharrar), where the team monitored posts and applicant interactions over a two-week period; (2) informal interviews with five university students who have previously applied to jobs through these channels, focusing on the pain points they experienced; and (3) a competitive analysis of formal job portals (EthioJobs, Ezega Jobs) and international Telegram-based job bots to identify best practices and gaps. The resulting requirements are categorised as functional (what the system must do) and non-functional (how well the system must do it), with supplementary specifications capturing the implicit rules and constraints that shape the system's behaviour.

PART ONE: SYSTEM REQUIREMENT SPECIFICATION

2.2 Description of the Current System

The current system for matching university students with part-time employment in Ethiopia consists of a loose collection of informal Telegram channels operated by individual entrepreneurs. The largest of these, Afriwork (Freelance Ethiopia), operates a primary channel with over fifty thousand subscribers and an associated bot (@freelanceethbot) that accepts job postings from verified employers and forwards them to the channel. Yegna Tutors Harrar is a smaller, regionally-focused channel that specialises in tutoring jobs in the Harar and Dire Dawa areas, operated through the bot @Yegnatutorharrar. Other smaller

channels exist for specific niches such as graphic design, content writing, and software development.

The workflow of the current system is as follows. An employer who wishes to post a job composes a free-text message containing the job title, subjects, location, schedule, salary, and contact information, then forwards this message to the channel operator's bot. The operator manually reviews the post (sometimes), reformats it for consistency, and publishes it to the channel. Interested students then forward the post along with their CV and contact details to the employer's personal Telegram account. The employer reviews the forwarded applications in their private chat, contacts promising candidates directly, and arranges interviews or trial sessions. Payments, when required, are made directly between the employer and the channel operator via Telebirr, with no formal record kept by either party. Several weaknesses of this current system are immediately apparent. Posts are unstructured, making them difficult to filter or search. There is no centralised archive; once a post scrolls out of the recent feed, it is effectively lost. Neither employers nor students are verified, leading to frequent scams and no-shows. The application process is highly manual and repetitive, requiring students to forward their CV multiple times. There is no tracking of which jobs a student has applied to or which applicants an employer is considering. Payments are informal and lack an audit trail. The channels have no monetisation model beyond optional donations, limiting their ability to grow into sustainable businesses. These weaknesses collectively motivate the development of Temariware as a structured, verified, and payment-enabled alternative.

2.3 Functional Requirements of the Proposed System

The functional requirements describe what the Temariware system must do. They are organised by user role and presented in Table 2.1 below. Each requirement is given a unique identifier (FR-XX) for traceability throughout the design and implementation phases.

Table 2.1: Functional Requirements

ID	Requirement
FR-01	System shall authenticate users via Telegram WebApp initData HMAC-SHA256 validation
FR-02	System shall allow any user to browse approved jobs without authentication
FR-03	System shall allow users to filter jobs by subject, location, type, gender, and search query

ID	Requirement
FR-04	System shall allow students to upload National ID, University ID, and grade report with client-side compression
FR-05	System shall block applications from students with incomplete profiles
FR-06	System shall allow verified employers to post jobs with all metadata fields
FR-07	System shall require payment screenshot upload when $\text{jobPostFee} > 0$
FR-08	System shall allow administrators to approve or reject pending jobs with reason
FR-09	System shall broadcast approved jobs to @TEMARIWARE channel automatically
FR-10	System shall push job alerts to subscribers whose category matches the job type
FR-11	System shall allow employers to view applicant profiles including documents
FR-12	System shall allow employers to accept or reject applicants
FR-13	System shall notify students via Telegram when accepted, requesting payment
FR-14	System shall allow students to upload payment screenshots after acceptance
FR-15	System shall allow administrators to approve or reject payment screenshots
FR-16	System shall mark applicants as HIRED upon payment approval
FR-17	System shall auto-close jobs when filledPositions reaches $\text{positionsAvailable}$
FR-18	System shall allow administrators to configure fees and payment instructions
FR-19	System shall support English, Amharic, and Afaan Oromoo with runtime switching
FR-20	System shall handle bot slash commands: /start, /latest, /search, /subscribe, /unsubscribe, /myapps, /postjob, /admin, /help

2.4 Non-Functional Requirements (Technical Requirements)

The non-functional requirements describe how well the Temariware system must perform the functions defined above. They are organised into seven categories: performance, security, usability, reliability, scalability, maintainability, and compatibility. Each

requirement is given a unique identifier (NFR-XX) and is testable through specific acceptance criteria.

Table 2.2: Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API response time < 500ms for 95% of requests; Mini App first contentful paint < 2s on 3G
NFR-02	Security	All API requests authenticated via HMAC-SHA256; no plaintext storage of payment data; HTTPS only
NFR-03	Usability	Mini App usable on 360px-wide screens; 3 languages; touch targets ≥ 44 px; haptic feedback on actions
NFR-04	Reliability	99% uptime (Vercel SLA); graceful error handling with user-friendly messages; no silent failures
NFR-05	Scalability	Serverless architecture scales to 1000 concurrent users on free tier; DB connection pooling via Neon
NFR-06	Maintainability	TypeScript throughout; modular components; Prisma typed queries; 100% lint-clean; documented in this report
NFR-07	Compatibility	Works in Telegram mobile (iOS, Android), Telegram Desktop, and Telegram Web; modern browser fallback

2.5 Supplementary Specification

This section documents the implicit rules, constraints, and anticipated changes that shape the Temariware system but do not fit neatly into the functional or non-functional requirement categories.

2.5.1 Business Rules

The following business rules govern the behaviour of the system:

- BR-01: Only users with an active Telegram account can use the system. There is no email-password or social-login alternative.

- BR-02: Only verified employers (verification status = APPROVED) or administrators can post new jobs. Unverified users must submit a verification request first.
- BR-03: Students cannot apply to any job until their profile is marked complete (National ID number + image, University ID number + image, last semester grade report image all present).
- BR-04: A student can apply to a given job only once. Duplicate applications are rejected at the API level via a unique constraint on (jobId, userId).
- BR-05: Job posting requires payment of the configured jobPostFee (default 500 ETB) via screenshot upload. The job remains in PENDING_PAYMENT status until the administrator approves both the content and the payment.
- BR-06: Application fees are per-job configurable by the employer. When the employer accepts an applicant, the student is notified to pay the applicationFee for that specific job (or the global default if not set).
- BR-07: An application cannot be marked HIRED until the student has uploaded a payment screenshot and the administrator has approved it.
- BR-08: When the number of hired applicants for a job reaches the positionsAvailable threshold, the job is automatically marked CLOSED and all remaining pending or accepted applications are marked CLOSED with notifications sent to all affected users.
- BR-09: Administrators can configure global fees, payment instructions, and Telebirr/CBE account numbers through the Settings tab. Changes take effect immediately for all subsequent transactions.
- BR-10: Every approved job is automatically broadcast to the @TEMARIWARE channel as a richly formatted card, and to all subscribers whose alert category matches the job's work type or who have subscribed to the ALL category.

2.5.2 Constraint

The following constraints limit the design and implementation choices:

- CON-01: The system must operate entirely on free-tier infrastructure to ensure zero recurring cost. This limits database storage to 500MB on Neon and serverless function execution time to 60 seconds on Vercel.
- CON-02: Payment verification must be screenshot-based in v1 because the project team does not have a registered business entity required for Chapa or Telebirr API integration. This constraint will be lifted in v2.

- CON-03: Uploaded images (documents and payment screenshots) must be stored as base64 strings inside the PostgreSQL database, with a maximum size of approximately 1MB per image after client-side compression. This is a trade-off to avoid the complexity of separate object storage.
- CON-04: The Mini App must work inside the Telegram WebApp container, which restricts certain browser APIs (no download attribute on links, limited clipboard access, no service workers).
- CON-05: The Telegram Bot API does not allow web_app inline buttons in channel posts; only URL buttons are permitted. This constraint shapes the channel broadcast UX.
- CON-06: The system must support three languages (English, Amharic, Afaan Oromoo) with runtime switching, requiring all UI strings to be externalised in a translation dictionary.

2.5.3 Change Case

The following change cases represent anticipated future modifications that the design should accommodate without major rework:

- CC-01: Add Chapa or Telebirr API integration for automated payment verification, replacing the screenshot-based flow. The existing Payment model already includes a chapaEnabled flag and chapaPublicKey/chapaSecretKey fields to support this.
- CC-02: Migrate image storage from base64-in-PostgreSQL to Supabase Storage or AWS S3. The User and Job models already separate image fields, making this migration straightforward.
- CC-03: Add an in-app messaging system between students and employers, replacing the current “contact via Telegram @username” approach.
- CC-04: Add automated scraping and import of jobs from existing Telegram channels such as Afriwork, to seed the platform during the early growth phase.
- CC-05: Add an employer dashboard with multi-job management, analytics, and bulk applicant actions.
- CC-06: Add saved/bookmarked jobs for students, with optional daily or weekly digest notifications.

2.5.4 Requirement Elicitation Models

This subsection presents the models used during requirements elicitation to capture and communicate the system's behaviour before the formal analysis in Part Two. Four models

are presented: essential use case modelling, CRC modelling, essential user interface prototyping, and user interface flow diagramming.

2.5.4.1 Essential Use Case Modeling

An essential use case model captures the interactions between the system's primary actors and the system itself, at a technology-independent level. The primary actors in Temariware are the Student, the Employer, the Administrator, and the Telegram Bot (which acts as both an actor and a system component). The essential use cases identified during elicitation are summarised below, and the diagram is presented in Figure 2.1.

- UC-01: Browse Job Feed — any user (including unauthenticated) can browse the list of approved jobs with filters.
- UC-02: View Job Detail — any user can view the full details of a specific job.
- UC-03: Register / Authenticate — a Telegram user is automatically registered and authenticated when they open the Mini App via the Telegram WebApp initData flow.
- UC-04: Complete Profile — a student uploads their personal information and mandatory documents.
- UC-05: Apply to Job — a student with a complete profile submits an application to an approved, open job.
- UC-06: Subscribe to Alerts — a user subscribes to receive push notifications when matching jobs are posted.
- UC-07: Post Job — a verified employer submits a new job posting with payment screenshot.
- UC-08: Verify Employer — an administrator reviews and approves or rejects an employer verification request.
- UC-09: Approve Job — an administrator reviews a pending job (content + payment) and approves or rejects it.
- UC-10: Review Applicants — an employer views the list of applicants for their jobs with full profiles and documents.
- UC-11: Accept Applicant — an employer accepts an applicant, triggering a payment request notification.
- UC-12: Upload Payment — a student uploads a payment screenshot after being accepted.
- UC-13: Approve Payment — an administrator verifies a payment screenshot and marks the applicant as HIRED.

- UC-14: Configure Settings — an administrator updates fees, payment instructions, and account numbers.
- UC-15: Broadcast to Channel — the system automatically posts approved jobs to the @TEMARIWARE channel.

Figure 2.1: Essential Use Case Diagram of Temariware

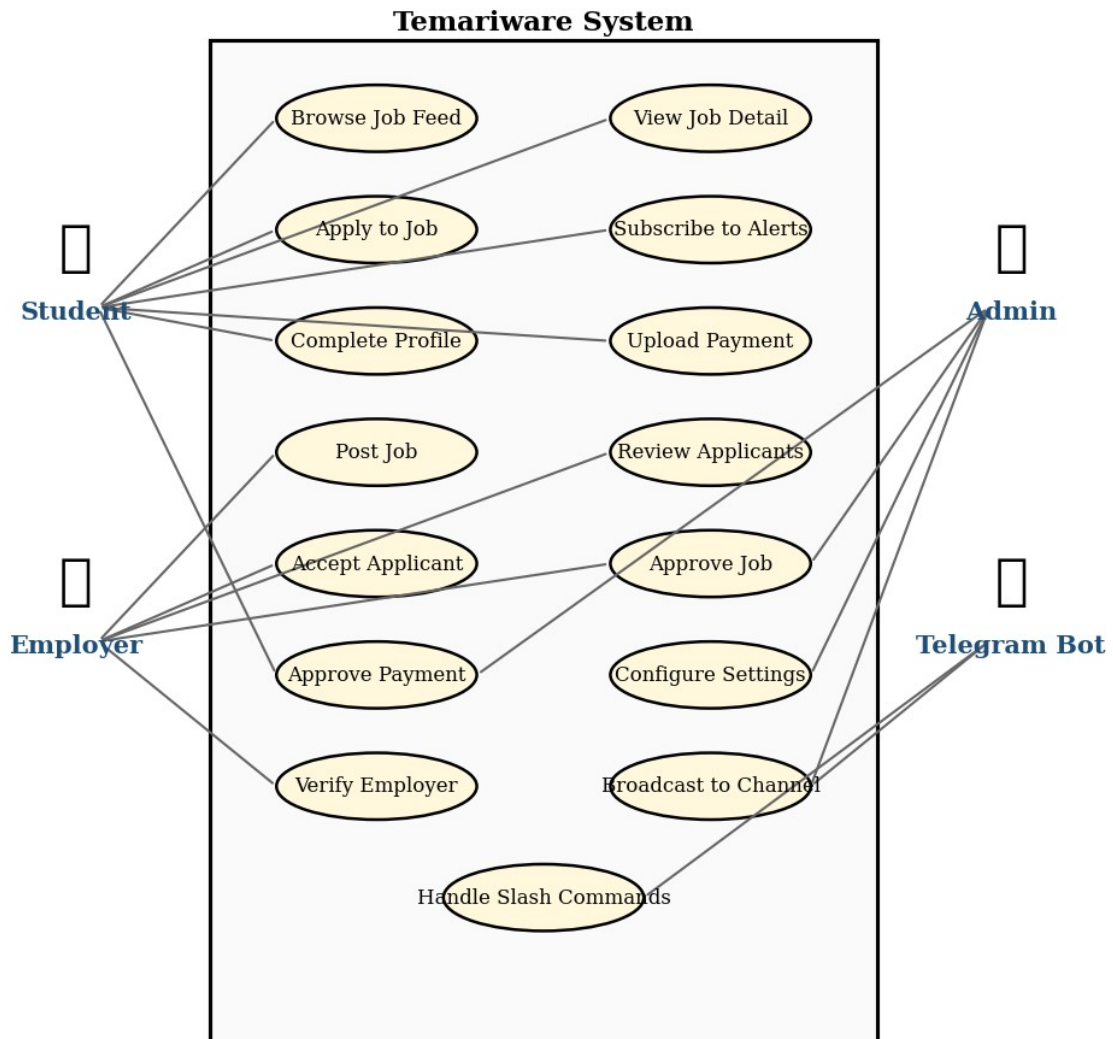


Figure 2.1: Essential Use Case Diagram of Temariware

2.5.4.2 CRC Modeling

Class-Responsibility-Collaboration (CRC) modelling is used during the conceptual design phase to assign responsibilities to each class and identify the collaborations between them. A CRC card is a physical or virtual index card listing the class name, its responsibilities, and the classes it collaborates with. Three representative CRC cards are presented below; the complete set is included in Appendix A.

Table 2.3: CRC Card — User Class

Class: User	
Responsibilities: • Store Telegram identity, role, language, and profile fields • Maintain mandatory documents (National ID, University ID, grade report) • Track profile completion status • Track employer verification status • Maintain alert subscription preferences	Collaborators: • Job (as employer) • Application (as applicant) • Subscription • VerificationRequest • Payment

Table 2.4: CRC Card — Job Class

Class: Job	
Responsibilities: • Store job metadata (title, subjects, location, salary, schedule, etc.) • Track status (PENDING_PAYMENT, PENDING, APPROVED, REJECTED, CLOSED) • Track positionsAvailable and filledPositions • Store per-job applicationFee override • Store payment screenshot and payment verification status	Collaborators: • User (as employer) • Application • Payment • Setting (for default fees)

Table 2.5: CRC Card — Application Class

Class: Application	
Responsibilities: • Link a student to a job with a status lifecycle • Store application message and payment screenshot • Track status: PENDING → ACCEPTED → PAYMENT_PENDING → HIRED (or REJECTED/CLOSED) • Trigger notifications on status transitions	Collaborators: • User (student) • Job • Payment

2.5.4.3 Essential User Interface (UI) Prototype

An essential UI prototype was created using low-fidelity wireframes to validate the information architecture and navigation flow before high-fidelity implementation. The prototype consists of six primary screens: Home (job feed with search and filters), Job Detail (full job information with apply button), Post Job (form with payment screenshot upload), Applications (tabs for “my applications” as student and “applicants to my jobs” as employer), Profile (with mandatory document upload), and Admin (tabs for pending jobs, payments, verifications, settings, stats). The wireframes are presented in Figure 2.5 and the high-fidelity implementation screenshots are included in Appendix B.

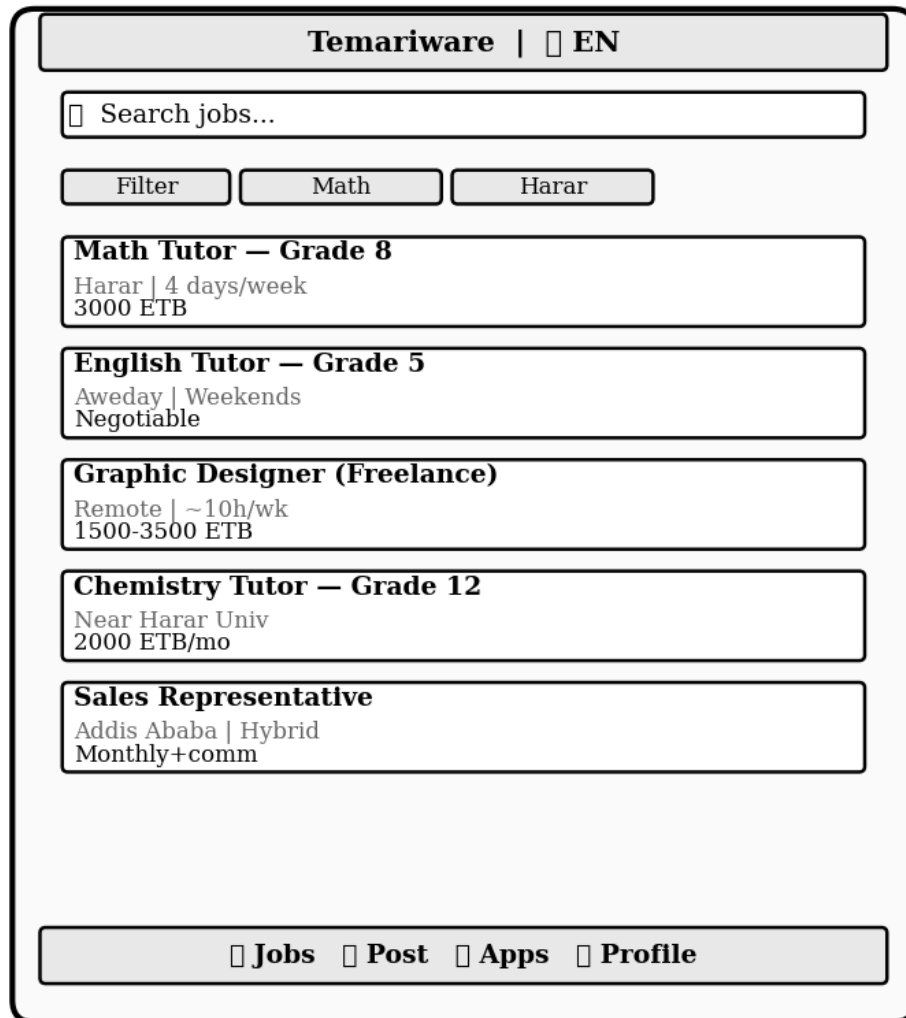
Figure 2.5: Essential UI Prototype — Home View

Figure 2.5: Essential User Interface Prototype — Home View wireframe

2.5.4.4 User Interface (UI) Flow Diagram

The UI flow diagram captures how a user navigates between the six primary screens of the Mini App. From the Home screen, a user can tap any job card to enter the Job Detail screen, or use the bottom navigation bar to switch to Post Job, Applications, Profile, or Admin (admin tab visible only to administrators). The Job Detail screen has a back button that returns to Home. The Post Job screen redirects to Home after successful submission. The Applications screen has two internal tabs (Applicants / My Applications) for employer users. The Profile screen includes a verification form that appears when the user is not yet verified. The UI flow diagram is presented in Figure 2.6.

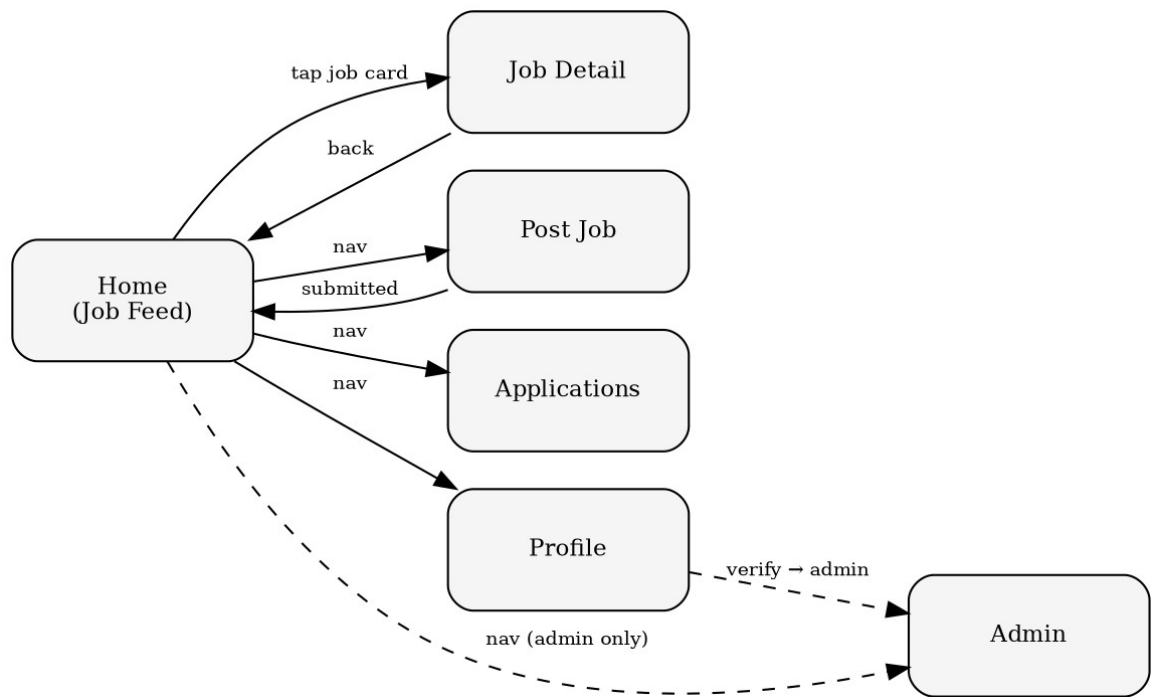


Figure 2.6: User Interface Flow Diagram

PART TWO: SYSTEM ANALYSIS

2.6 The Proposed System Models (in OO Approach)

Part Two of this chapter presents the system models for Temariware using the Object-Oriented approach. These models — system use case model, conceptual class model, sequence diagram, activity diagram, and user interface prototype — form the bridge between the requirements captured in Part One and the detailed design that will be presented in Chapter Three.

2.6.1 System Use Case Model

The system use case model refines the essential use cases from Section 2.5.4.1 by adding system-level detail. The model is presented in Figure 2.7 and identifies four primary actors (Student, Employer, Administrator, Telegram Bot) and fifteen use cases. The Student actor is associated with Browse Job Feed, View Job Detail, Complete Profile, Apply to Job, Subscribe to Alerts, Upload Payment, and View My Applications. The Employer actor is associated with Post Job, Review Applicants, Accept Applicant, Reject Applicant, and Verify Employer. The Administrator actor is associated with Approve Job, Approve Payment, Approve Verification, and Configure Settings. The Telegram Bot actor is associated with Handle Slash Commands, Push Job Alerts, and Broadcast to Channel. Several include and extend relationships are modelled — for example, Apply to Job includes Check Profile Complete (a student cannot apply if their profile is incomplete), and Post Job extends Upload Payment Screenshot (the upload is only required when the jobPostFee is greater than zero).

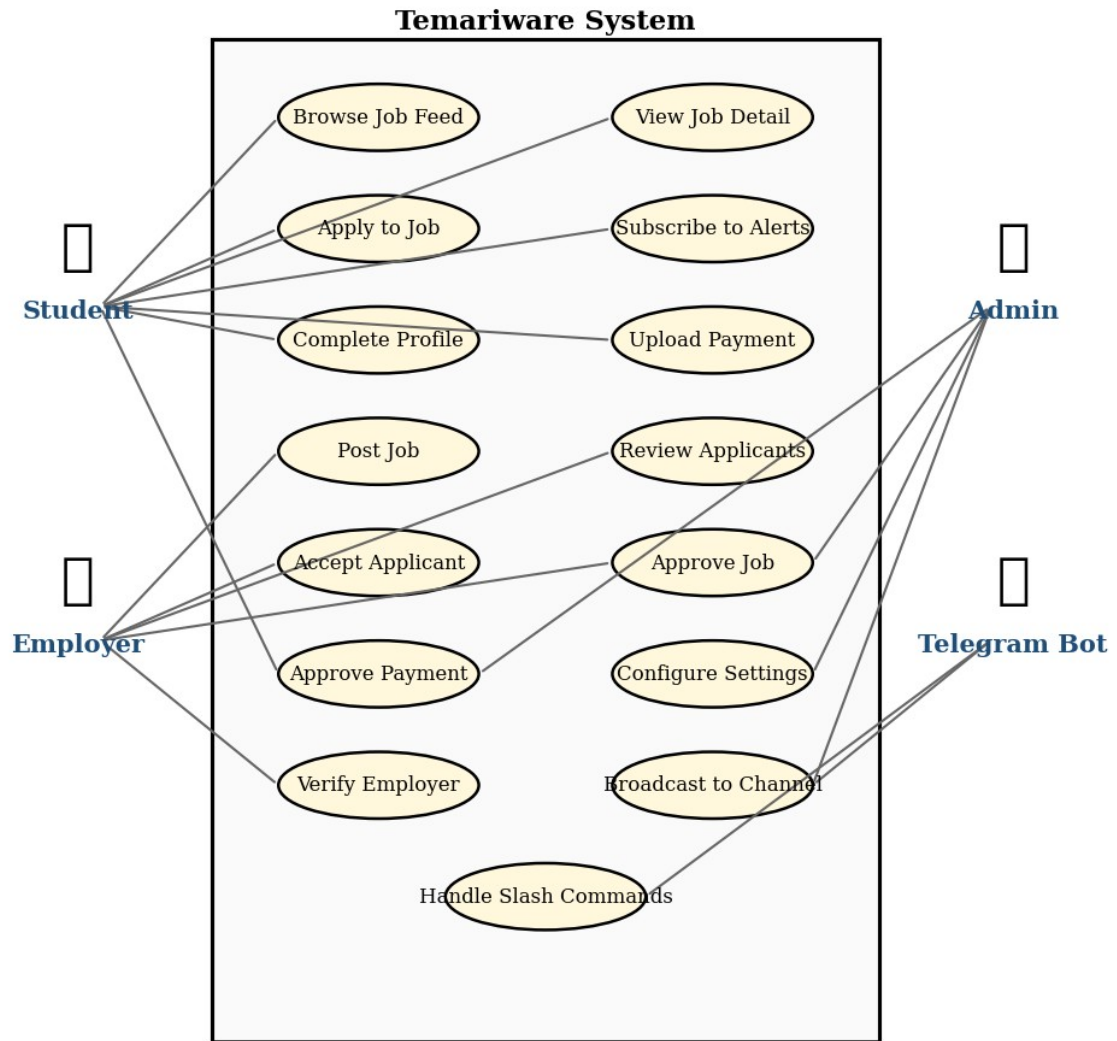
Figure 2.1: Essential Use Case Diagram of Temariware

Figure 2.7: System Use Case Model (refined)

2.6.2 Conceptual Class Model

The conceptual class model identifies the primary domain entities and their relationships at an analysis level, independent of implementation. The seven primary classes are User, Job, Application, Subscription, VerificationRequest, Setting, and Payment. A User has many Jobs (as employer), many Applications (as student), many Subscriptions, and may have one VerificationRequest. A Job has many Applications. An Application belongs to one User and one Job, and may have one associated Payment. A Setting is a singleton. The conceptual class diagram is presented in Figure 2.8.

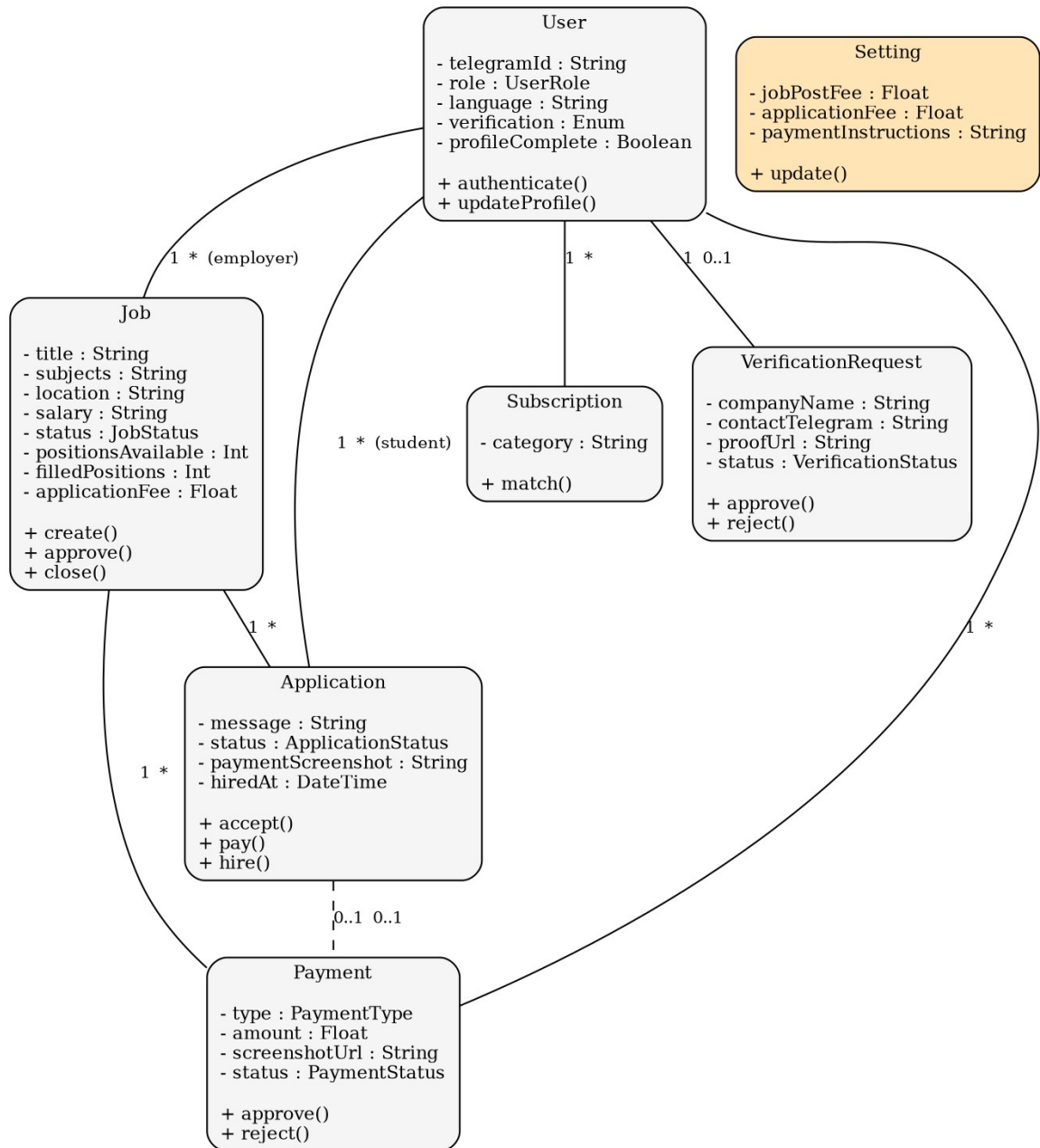


Figure 2.8: Conceptual Class Model

2.6.3 Sequence Diagram

Sequence diagrams capture the time-ordered interactions between objects for the system's most important flows. Three sequence diagrams are presented: Apply to Job (Figure 2.9), Accept Applicant and Pay (Figure 2.10), and Approve Job and Broadcast (Figure 2.11). The Apply to Job flow involves the Student (Mini App), the Jobs API, the Applications API, the Database, and the Employer (via Telegram notification). The Accept Applicant and Pay flow involves the Employer, the Accept API, the Student (via Telegram notification), the Pay API, the Admin, and the Approve Payment API. The Approve Job and Broadcast flow

involves the Admin, the Approve API, the Channel (via Telegram Bot API), and the Subscribers (via Telegram push).

Figure 2.3: Sequence Diagram — Apply to Job

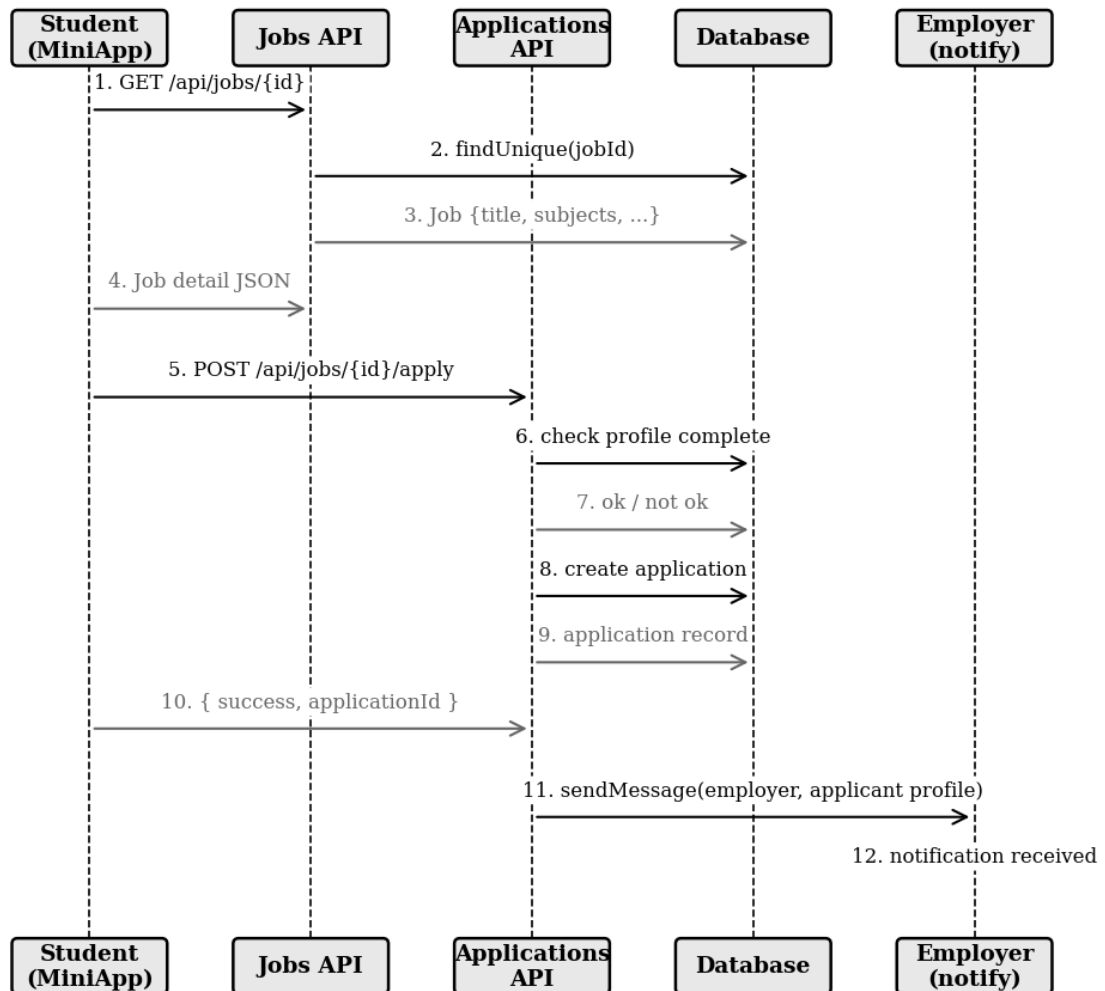


Figure 2.9: Sequence Diagram — Apply to Job flow

Figure 2.10: Sequence Diagram — Accept Applicant and Pay

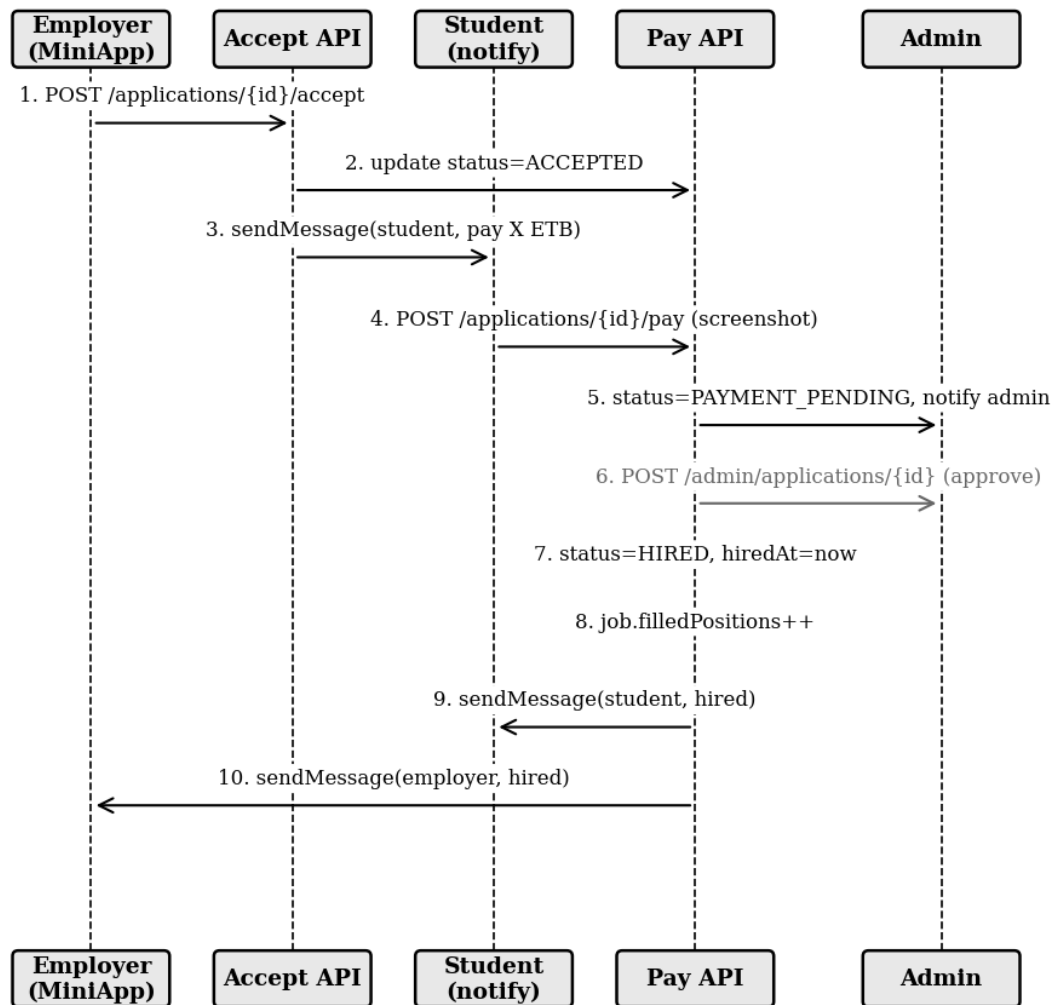


Figure 2.10: Sequence Diagram — Accept Applicant and Pay flow

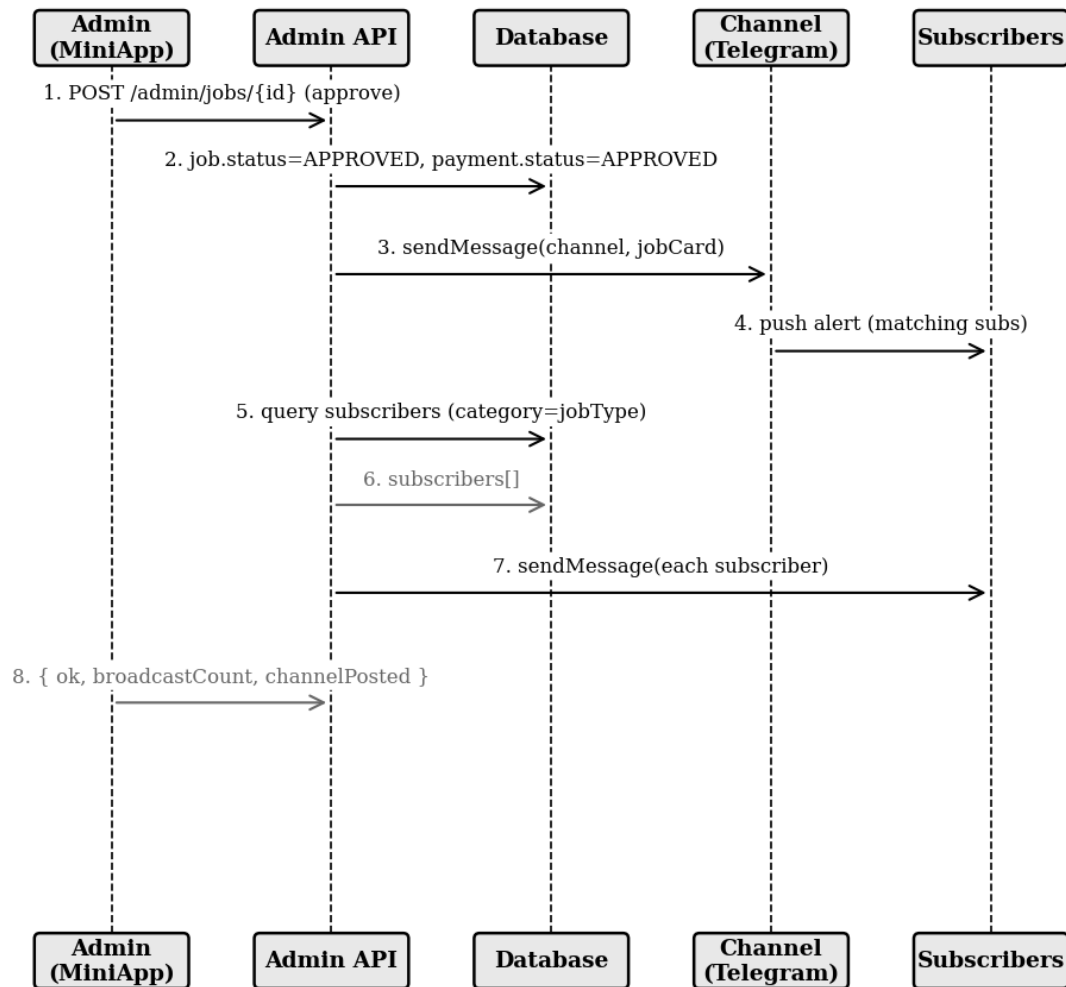
Figure 2.11: Sequence Diagram — Approve Job and Broadcast

Figure 2.11: Sequence Diagram — Approve Job and Broadcast flow

2.6.4 Activity Diagram

Activity diagrams capture the parallel and conditional flows within the system. The most important activity diagram is the Job Posting and Approval Flow (Figure 2.12), which shows the path from an employer submitting a job to the job being broadcast to the channel and alerts being sent to subscribers. The flow includes decision points for payment verification, content review, and broadcast success, with appropriate error paths. A second activity diagram captures the Application Lifecycle (Figure 2.13), showing the states an application passes through from PENDING to HIRED or CLOSED, including the auto-close trigger when positions are filled.

Figure 2.4: Activity Diagram — Job Posting & Approval Flow

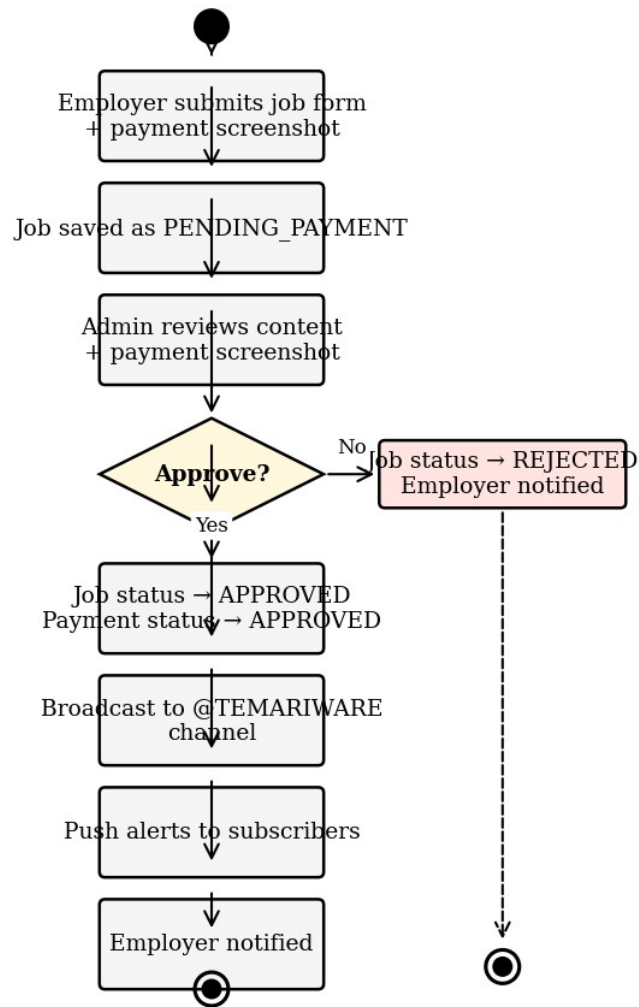


Figure 2.12: Activity Diagram — Job Posting and Approval Flow

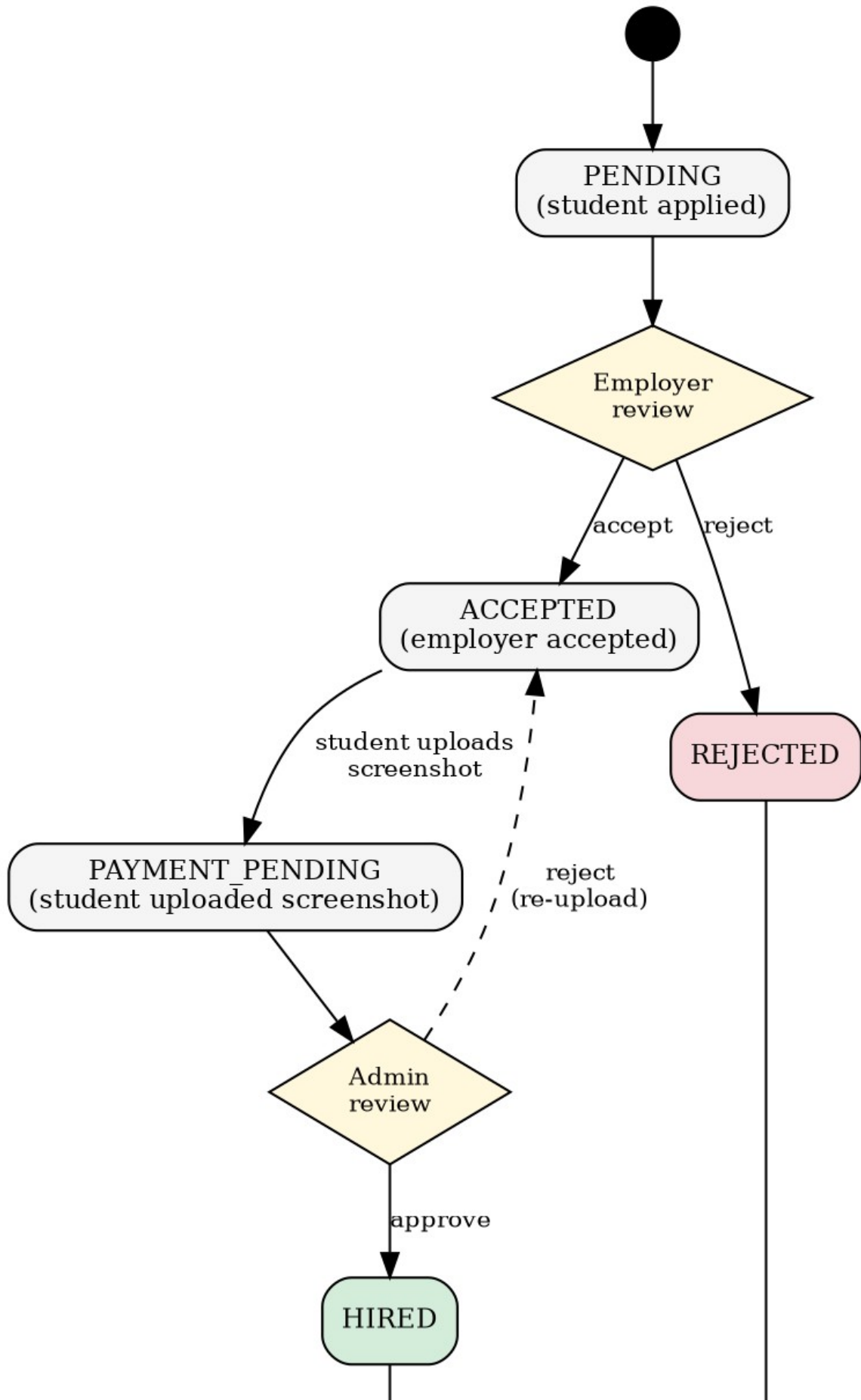


Figure 2.13: Activity Diagram — Application Lifecycle

2.6.5 User Interface Prototype

The high-fidelity user interface prototype was implemented directly in React with Tailwind CSS and shadcn/ui components, following a Telegram-native visual design system. The prototype consists of six views: Home (job feed with debounced search and filter chips), Job Detail (structured facts grid with apply form), Post Job (full form with payment screenshot upload), Applications (tabbed view for students and employers), Profile (with mandatory document upload and image compression), and Admin (tabbed view with pending jobs, payments, verifications, settings, and stats). Screenshots of the implemented prototype are included in Appendix B.

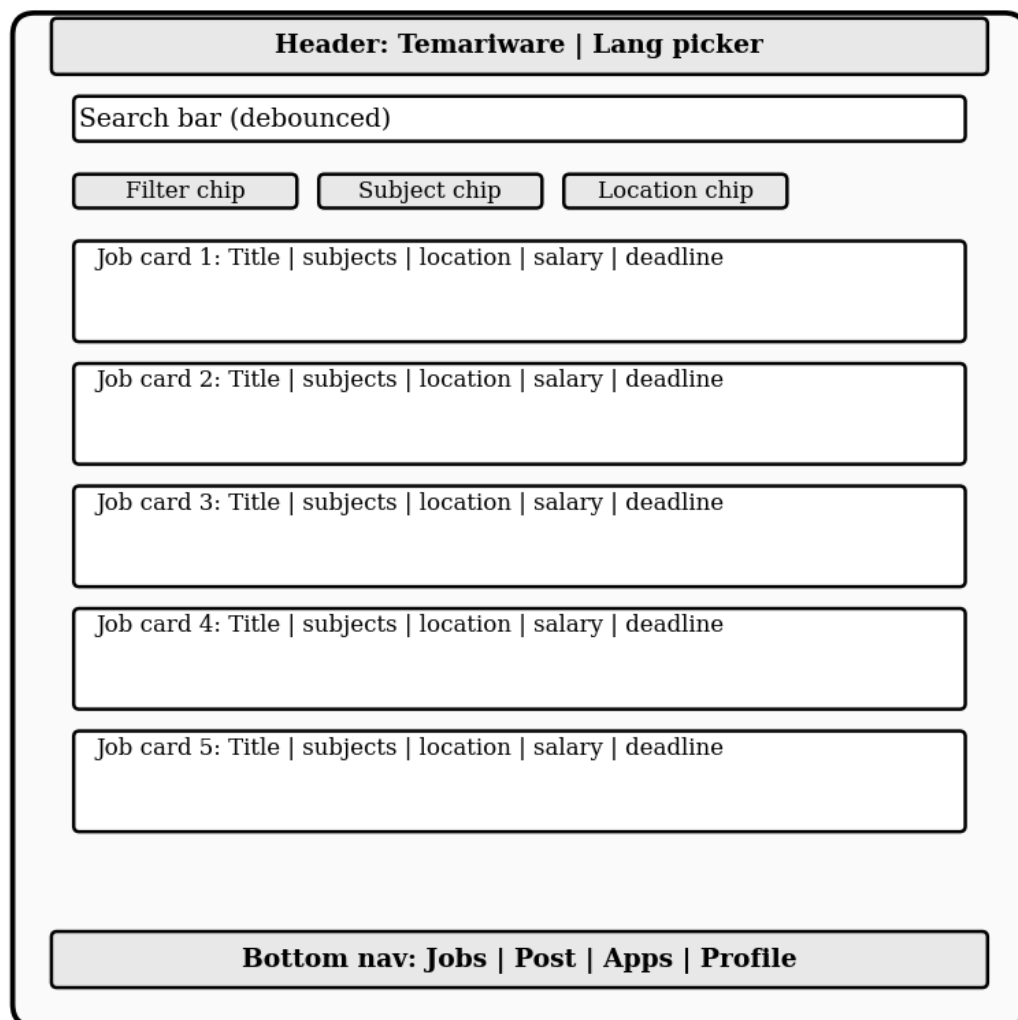
Figure 3.10: UI Diagram — Home View

Figure 2.14: User Interface Prototype — Home View (implemented)

Figure 3.11: UI Diagram — Job Detail View

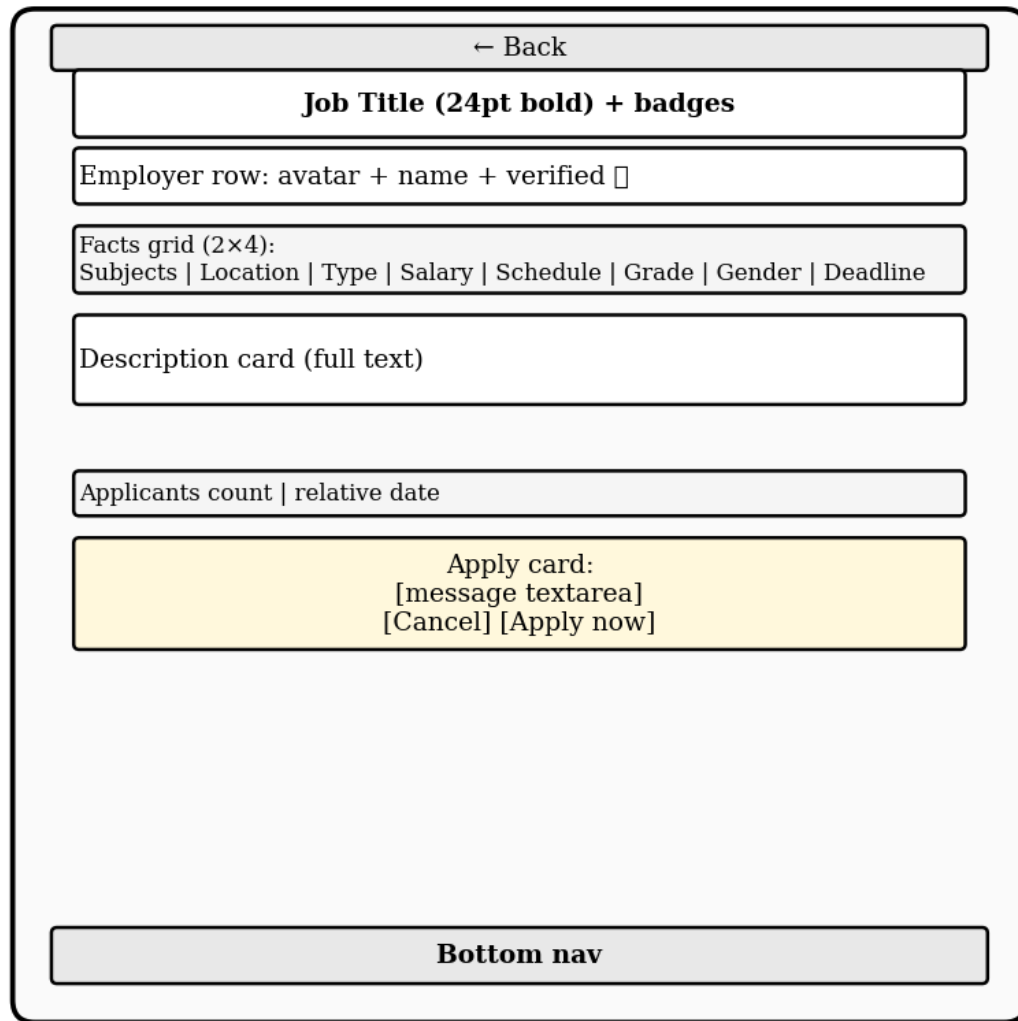


Figure 2.15: User Interface Prototype — Job Detail View (implemented)

Figure 3.15: UI Diagram — Admin View

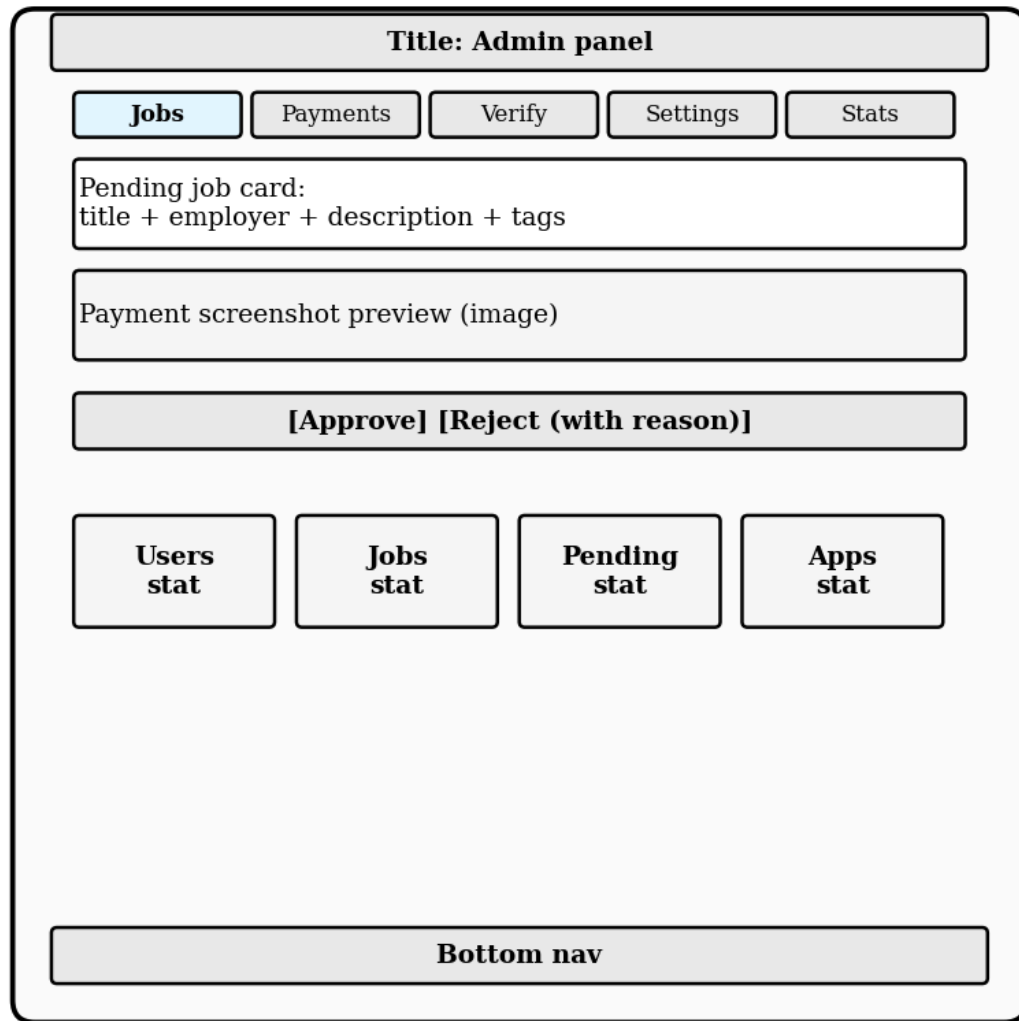


Figure 2.16: User Interface Prototype — Admin Panel (implemented)

CHAPTER THREE: DESIGN SPECIFICATION

3.1 Introduction/Overview

This chapter presents the design specification for the Temariware system, building on the requirements and analysis presented in Chapter Two. The chapter is organised into two main sections: Software Architecture (Section 3.2), which describes the high-level architectural style and presents the architecture using UML diagrams including class type architecture, component diagram, deployment diagram, and persistent diagram; and Detailed Design of the System (Section 3.3), which presents the design-level class diagram, collaboration diagram, state chart diagram, and user interface diagram. Together, these design artefacts provide a complete blueprint that can be handed off to the implementation team.

The design follows the Object-Oriented paradigm consistently with the analysis in Chapter Two. The architecture is a modern variant of the classic Model-View-Controller (MVC) pattern, adapted to the Next.js App Router conventions where API routes serve as controllers and React components serve as views. The persistence layer is managed by Prisma ORM, which generates type-safe query builders from the Prisma schema and abstracts away the underlying PostgreSQL database.

3.2 Software Architecture

3.2.1 Architectural Styles

Several architectural styles were considered during the design phase. The selected style is a Layered Architecture with four layers: Presentation (React components), Application (Next.js API routes), Domain (Prisma models and business logic), and Persistence (PostgreSQL database on Neon). This style was chosen because it provides clear separation of concerns, matches the conventions of the Next.js framework, and is well-understood by the development team. Alternative styles considered included the Microkernel pattern (too complex for the current scope), Event-Driven Architecture (would introduce unnecessary complexity for the current scale), and the Serverless-First pattern (effectively a deployment-time choice rather than an architectural style, and already adopted via Vercel). The comparison of architectural styles is summarised in Table 3.1.

Table 3.1: Software Architecture Styles Considered

Style	Pros	Verdict
Layered Architecture	Separation of concerns, matches Next.js conventions, well-understood	Selected
Microkernel	Highly extensible, supports plugins	Too complex for current scope
Event-Driven	Loosely coupled, scalable	Adds unnecessary complexity (Kafka, etc.)
Serverless-First	Auto-scaling, pay-per-use	Already adopted at deployment level

3.2.2 Describing an Architecture using UML

This subsection describes the Temariware architecture using four UML diagrams: class type architecture, component diagram, deployment diagram, and persistent diagram. Each diagram captures a different view of the system and together they provide a complete architectural picture.

3.2.2.1 Class Type Architecture

The class type architecture organises the system's classes into three types: Entity classes (User, Job, Application, Subscription, VerificationRequest, Setting, Payment) which represent persistent domain objects; Boundary classes (HomeView, JobDetailView, PostJobView, ApplicationsView, ProfileView, AdminView, BotWebhookHandler) which interact with external actors; and Control classes (AuthController, JobController, ApplicationController, PaymentController, AdminController, BotController) which orchestrate the business logic between entity and boundary classes. This separation follows the Entity-Boundary-Control (ECB) pattern, which is a refinement of MVC that is particularly well-suited to object-oriented systems with use-case-driven design. The class type architecture diagram is presented in Figure 3.1.

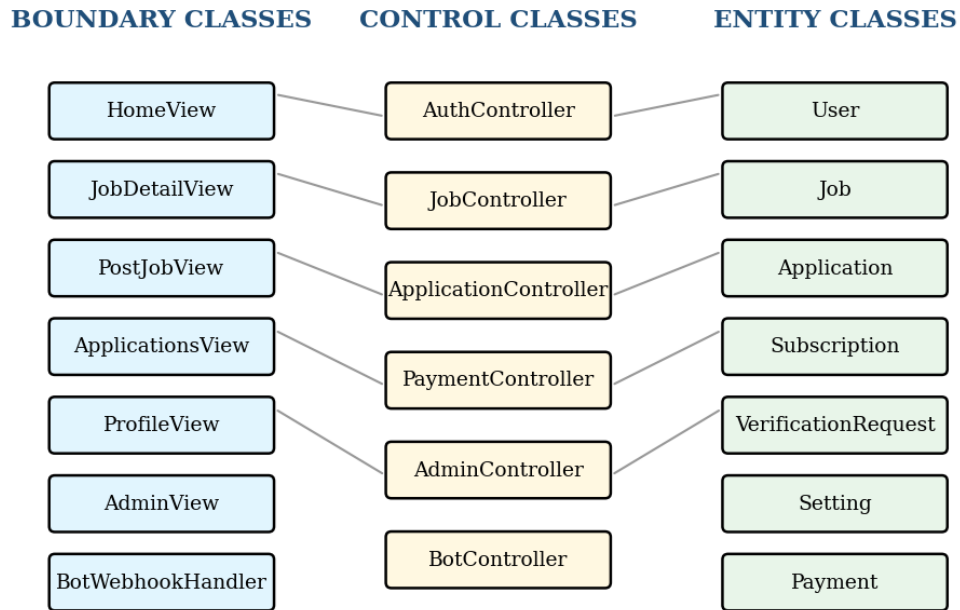
Figure 3.1: Class Type Architecture (Entity-Boundary-Control)

Figure 3.1: Class Type Architecture Diagram

3.2.2.2 Component Diagram

The component diagram shows the deployable software components and their dependencies. Temariware consists of five primary components: the Next.js Mini App (deployed on Vercel), the Telegram Bot Webhook (an API route within the Next.js app), the PostgreSQL Database (hosted on Neon), the Telegram Bot API (external, provided by Telegram), and the @TEMARIWARE Channel (external, owned by the project team). The Mini App component depends on the Database component (via Prisma ORM) and on the Telegram Bot API component (via HTTPS for sending messages and via initData validation for authenticating users). The Bot Webhook component depends on the Database

and on the Telegram Bot API. The Telegram Bot API pushes updates to the Bot Webhook via HTTPS webhook. The component diagram is presented in Figure 3.2.

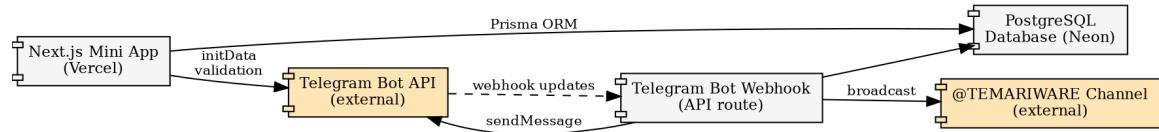


Figure 3.2: Component Diagram

3.2.2.3 Deployment Diagram

The deployment diagram shows the physical nodes on which the software components run and the communication paths between them. Temariware is deployed across three physical nodes: (1) the Vercel Edge Network, which runs the Next.js Mini App and Bot Webhook as serverless functions across multiple global regions; (2) the Neon Cloud, which runs the PostgreSQL database in the Frankfurt region (eu-central-1, selected for proximity to Ethiopia); and (3) the Telegram Cloud, which runs the Telegram Bot API and hosts the @TEMARIWARE channel. End users (students, employers, administrators) access the system through the Telegram client on their mobile or desktop devices, which communicates with the Telegram Cloud, which in turn communicates with the Vercel Edge Network via webhook and inline button URLs. The deployment diagram is presented in Figure 3.3.

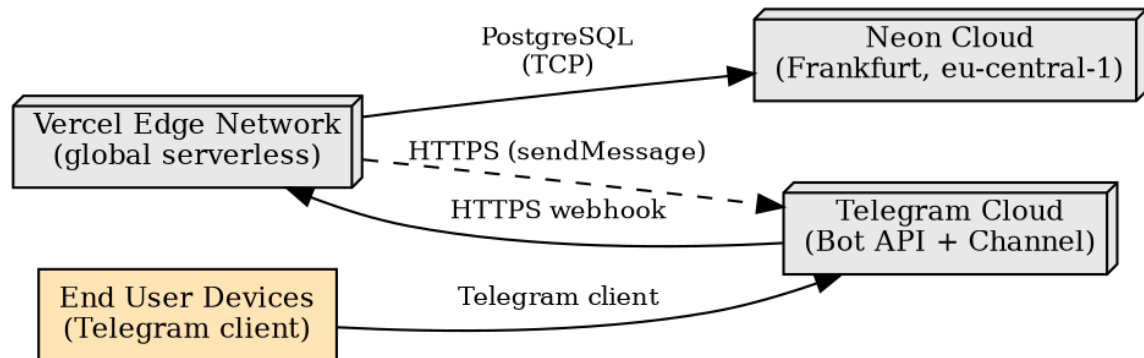


Figure 3.3: Deployment Diagram

3.2.2.4 Persistent Diagram

The persistent diagram (also known as the data model or entity-relationship diagram) shows the structure of the database. Temariware's database consists of seven tables: User, Job, Application, Subscription, VerificationRequest, Setting, and Payment. The User table is central, with one-to-many relationships to Job (as employer), Application (as student), Subscription, VerificationRequest, and Payment. The Job table has a one-to-many relationship with Application and Payment. The Application table has a unique constraint

on (jobId, userId) to prevent duplicate applications. The Setting table is a singleton (single row with id = 'singleton') that stores global configuration. The persistent diagram with full field-level detail is presented in Figure 3.4 and a summary of the tables is provided in Table 3.2.

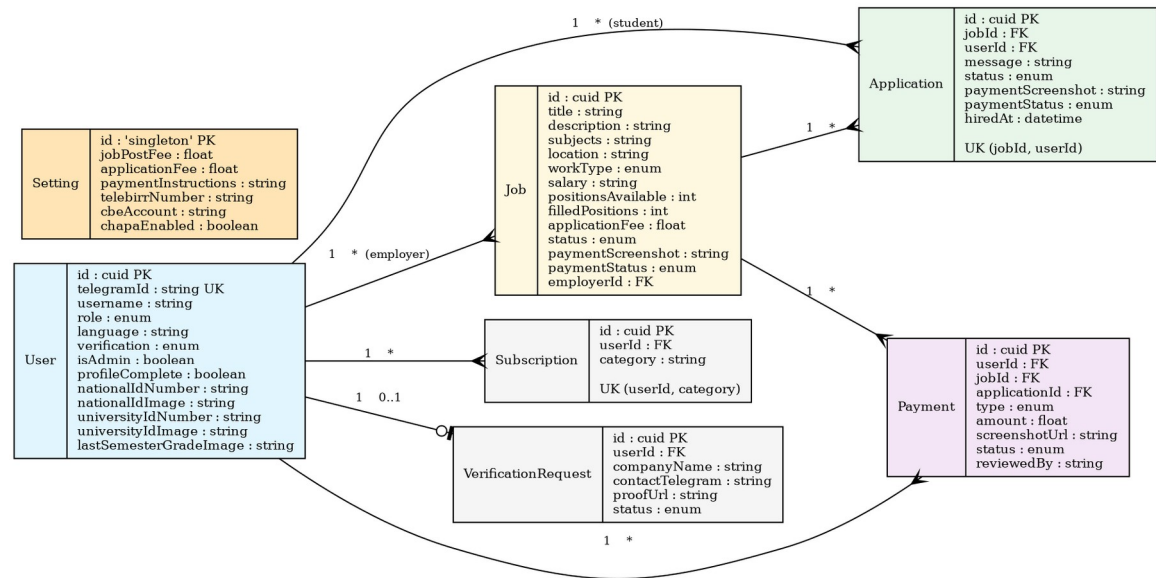


Figure 3.4: Persistent Diagram (Entity-Relationship Diagram)

Table 3.2: Persistent Data Model Summary

Table	Key Fields
User	id, telegramId, username, role, language, profile fields, document fields, verification, isAdmin
Job	id, title, description, subjects, location, workType, salary, positionsAvailable, filledPositions, applicationFee, status, payment fields
Application	id, jobId, userId, message, status, paymentScreenshot, paymentStatus, hiredAt, createdAt
Subscription	id, userId, category
VerificationRequest	id, userId, companyName, contactTelegram, proofUrl, status
Setting	id (singleton), jobPostFee, applicationFee, paymentInstructions, telebirrNumber, cbeAccount, chapaEnabled
Payment	id, userId, jobId, type, applicationId, amount, screenshotUrl, status, reviewedBy

3.3 Detailed Design of the System

This section presents the detailed design of the Temariware system, refining the conceptual models from Chapter Two into implementation-ready specifications. Four detailed design artefacts are presented: the design-level class diagram, the collaboration diagram, the state chart diagram, and the user interface diagram.

3.3.1 Design Level Class Diagram

The design-level class diagram refines the conceptual class diagram from Section 2.6.2 by adding attributes, operations, visibility, and data types. Each class now includes its full set of fields with types (matching the Prisma schema), key methods (matching the API route handlers), and visibility markers (public for API-exposed operations, private for internal logic). For example, the User class includes attributes such as id (cuid), telegramId (string, unique), role (enum), verification (enum), and the seven mandatory document fields; it includes operations such as upsertUser(), authenticate(), and updateProfile(). The complete design-level class diagram is presented in Figure 3.5.

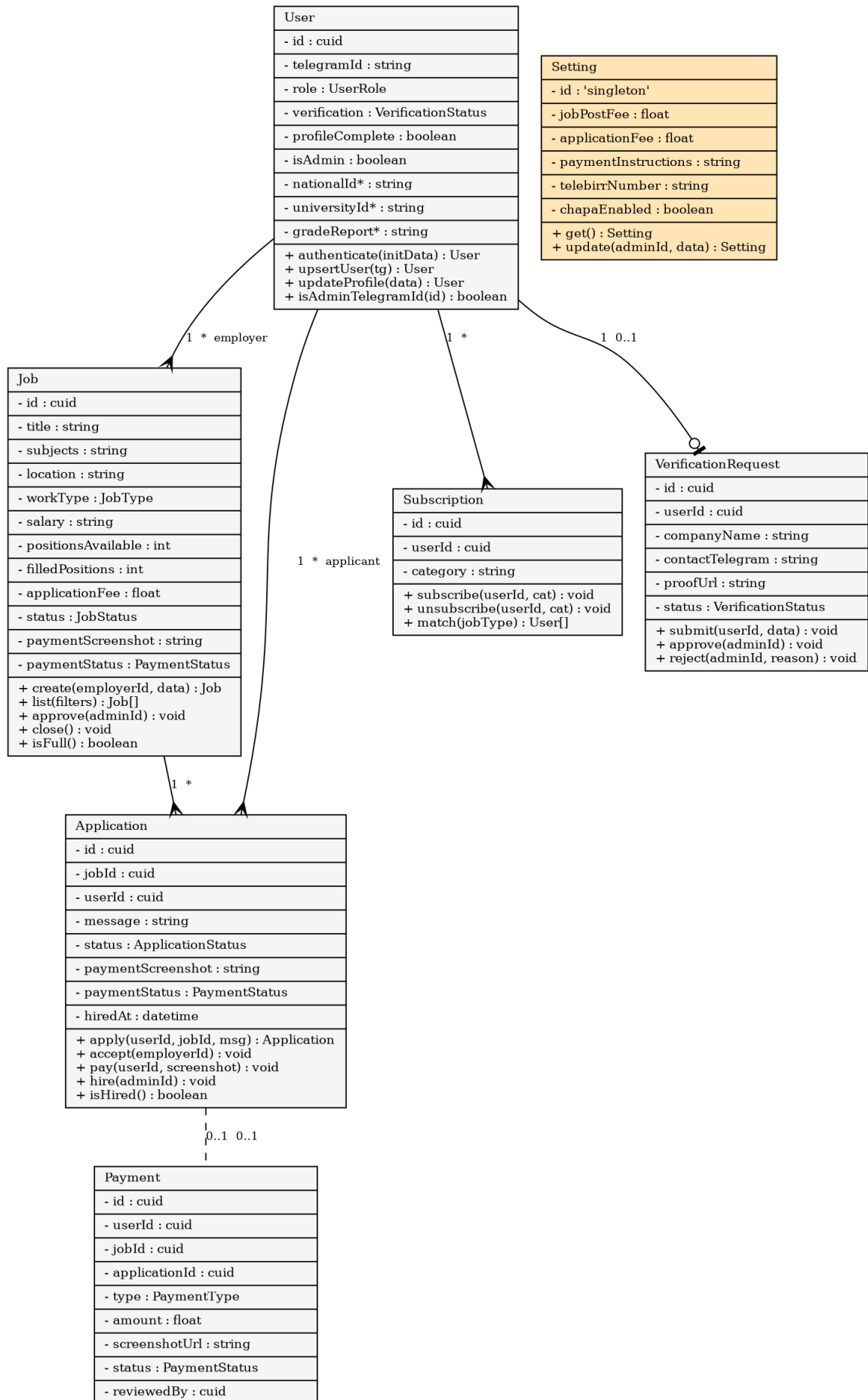


Figure 3.5: Design-Level Class Diagram

3.3.2 Collaboration Diagram

The collaboration diagram (also known as a communication diagram) shows the objects that collaborate to fulfil a specific use case, along with the messages passed between them. Three collaboration diagrams are presented: (1) Apply to Job (Figure 3.6), showing the collaboration between MiniApp, JobAPI, ApplicationAPI, Database, and EmployerNotification; (2) Post Job and Pay (Figure 3.7), showing the collaboration between EmployerMiniApp, JobsAPI, PaymentModel, Database, and AdminNotification; and (3) Approve Payment and Hire (Figure 3.8), showing the collaboration between AdminMiniApp, AdminAPI, ApplicationModel, JobModel, SubscriberNotification, and AutoCloseTrigger. The collaboration diagrams emphasise the structural relationships between objects while the sequence diagrams in Section 2.6.3 emphasise the time-ordering of messages.

Figure 3.6: Collaboration Diagram — Apply to Job

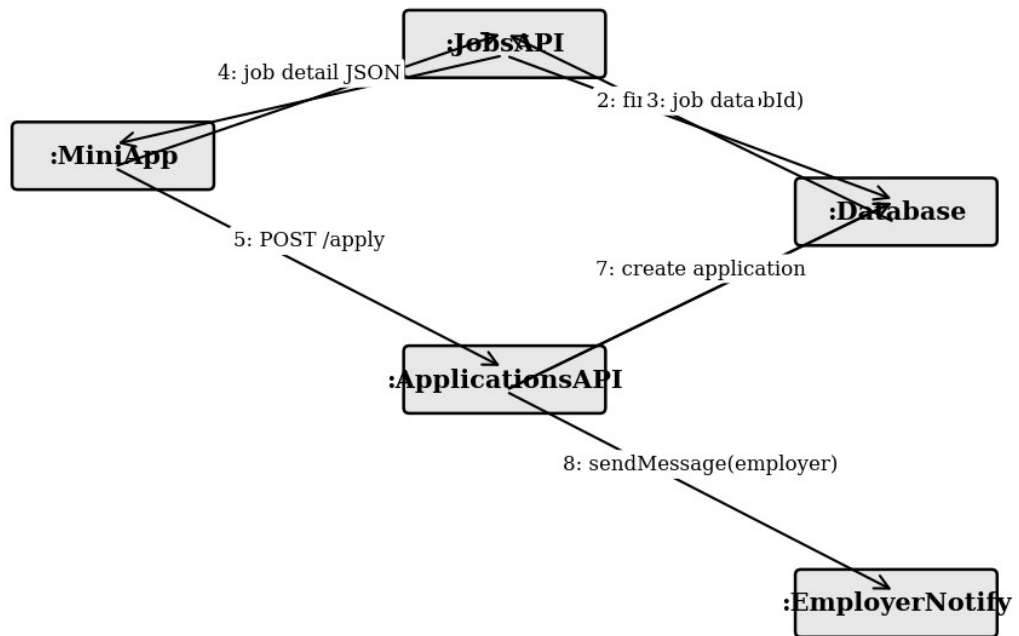


Figure 3.6: Collaboration Diagram — Apply to Job

Figure 3.7: Collaboration Diagram — Post Job and Pay

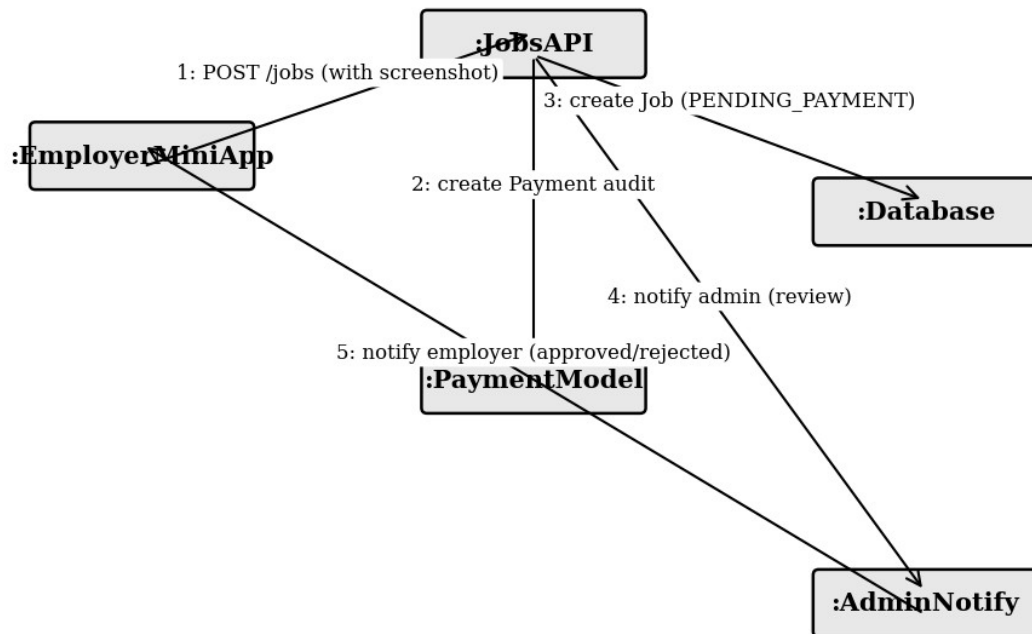


Figure 3.7: Collaboration Diagram — Post Job and Pay

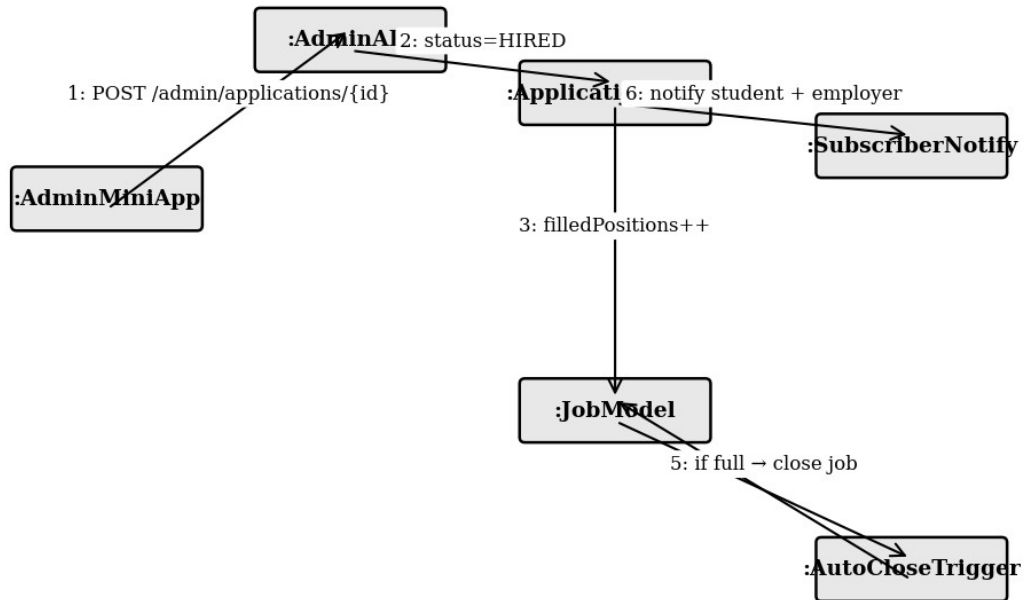
Figure 3.8: Collaboration Diagram — Approve Payment and Hire

Figure 3.8: Collaboration Diagram — Approve Payment and Hire

3.3.3 State Chart Diagram

The state chart diagram captures the lifecycle of the Application object, which is the most stateful entity in the system. An Application begins in the PENDING state when a student submits it. From PENDING, the employer can transition it to ACCEPTED (if they want to hire the student) or REJECTED (if they do not). From ACCEPTED, the student uploads a payment screenshot, transitioning the application to PAYMENT_PENDING. From PAYMENT_PENDING, the administrator can transition it to HIRED (if the payment is verified) or back to ACCEPTED (if the payment is rejected, allowing the student to re-upload). At any point before HIRED, the application can be transitioned to CLOSED if the job's positions are filled by other applicants. Once HIRED or CLOSED, no further transitions are possible. The state chart diagram is presented in Figure 3.9.

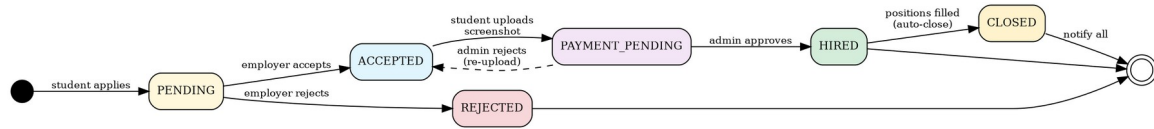


Figure 3.9: State Chart Diagram — Application Lifecycle

3.3.4 User Interface Diagram

The user interface diagram shows the structure of each Mini App view in terms of its constituent UI components and their layout. The Home view consists of a search bar, filter chips, a job feed list, and a bottom navigation bar. The Job Detail view consists of a back button, a job title with badges, an employer row with avatar, a facts grid, a description card, an apply button (or status card if already applied), and the bottom navigation bar. The Post Job view consists of a fee notice card, a form card with multiple input fields grouped in a two-column grid, a payment screenshot uploader, and a submit button. The Applications view consists of a tabbed layout (Applicants / My Applications) with cards showing applicant or application summaries, and a sliding sheet for full applicant profile review. The Profile view consists of an identity header, language picker, verification status card, profile form, university form, mandatory documents card with three image uploaders, and alerts form. The Admin view consists of a five-tab layout (Jobs, Payments, Verifications, Settings, Stats) with appropriate content per tab. The user interface diagrams are presented in Figures 3.10 through 3.15.

Figure 3.10: UI Diagram — Home View

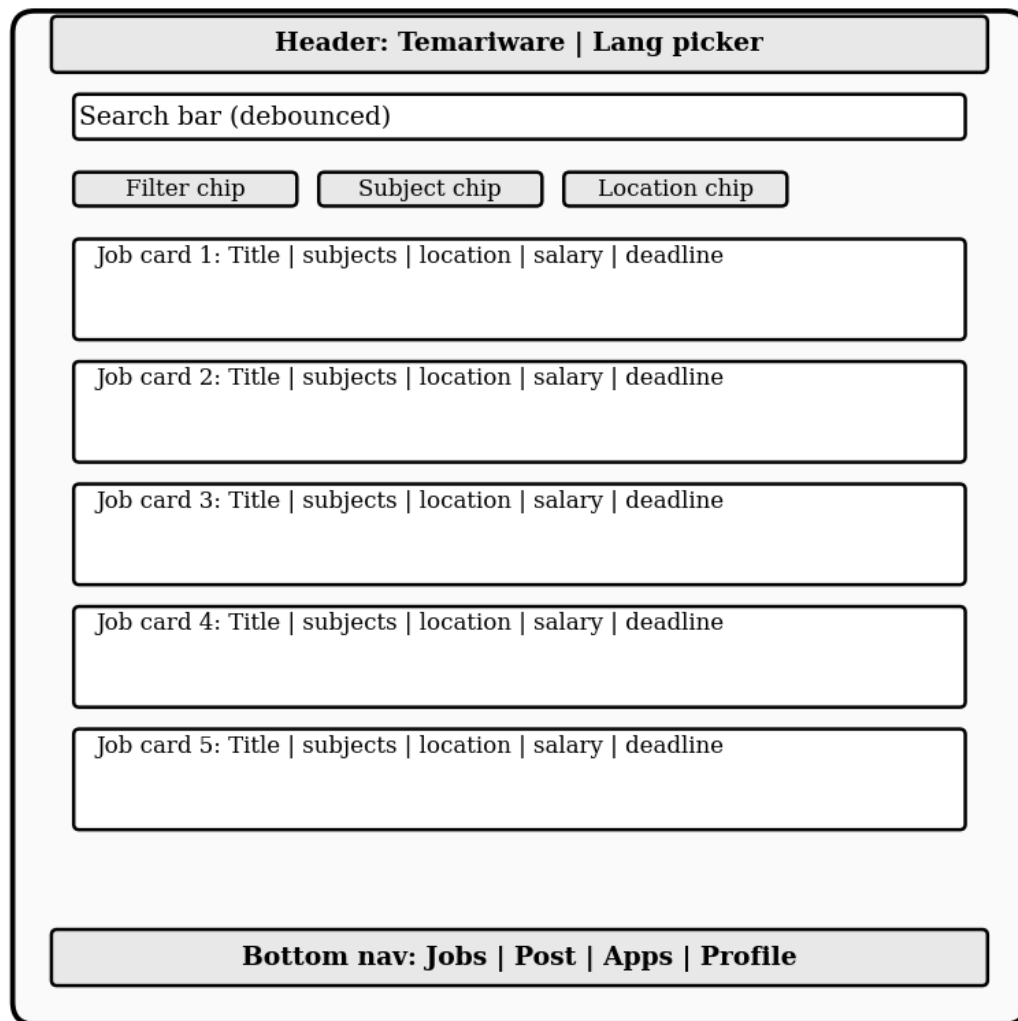


Figure 3.10: UI Diagram — Home View

Figure 3.11: UI Diagram — Job Detail View

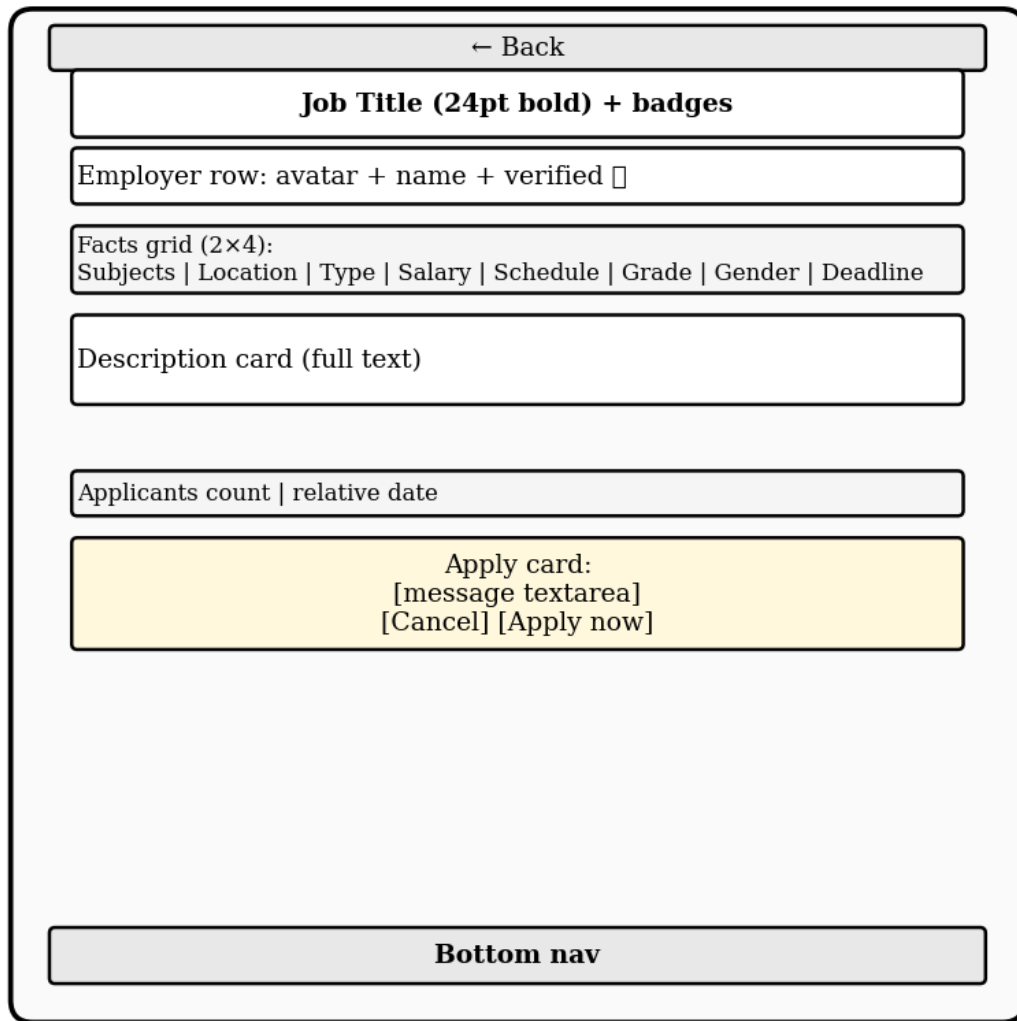


Figure 3.11: UI Diagram — Job Detail View

Figure 3.12: UI Diagram — Post Job View

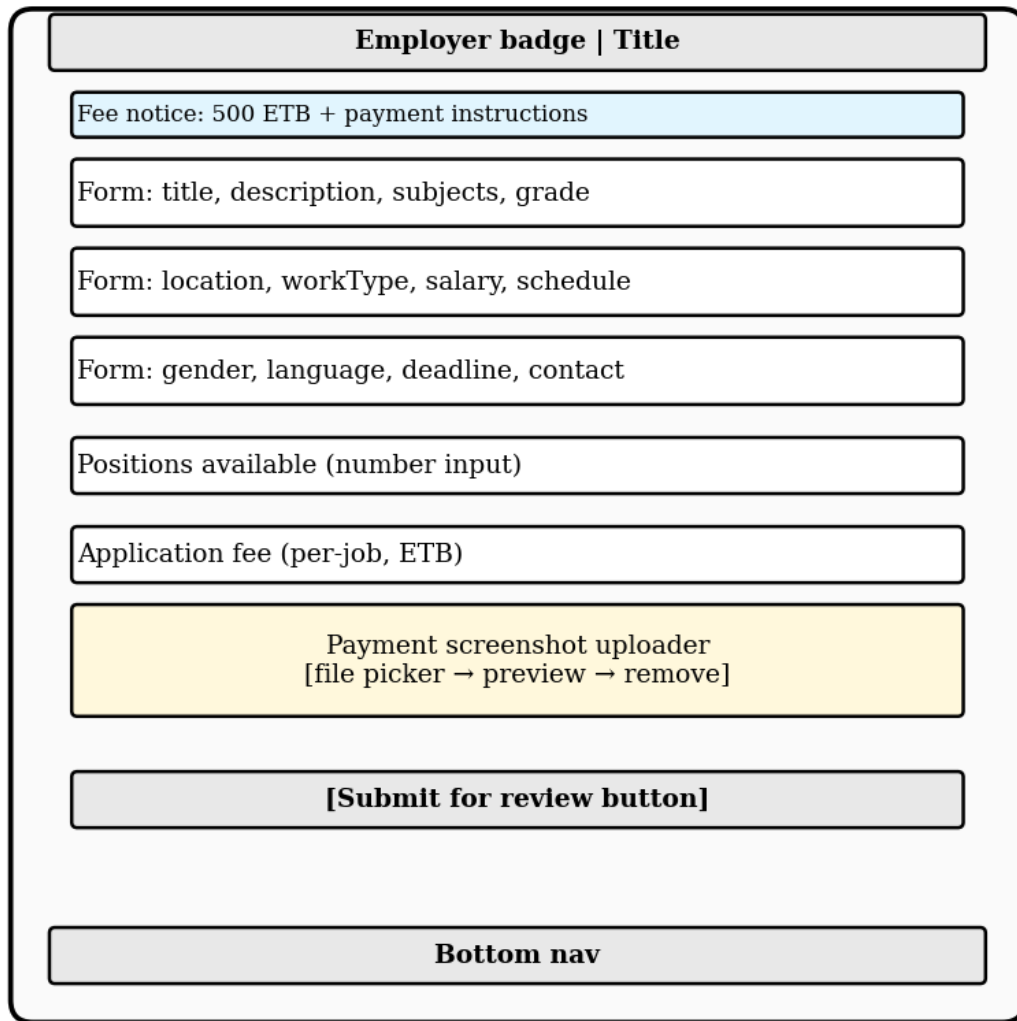


Figure 3.12: UI Diagram — Post Job View

Figure 3.13: UI Diagram — Applications View

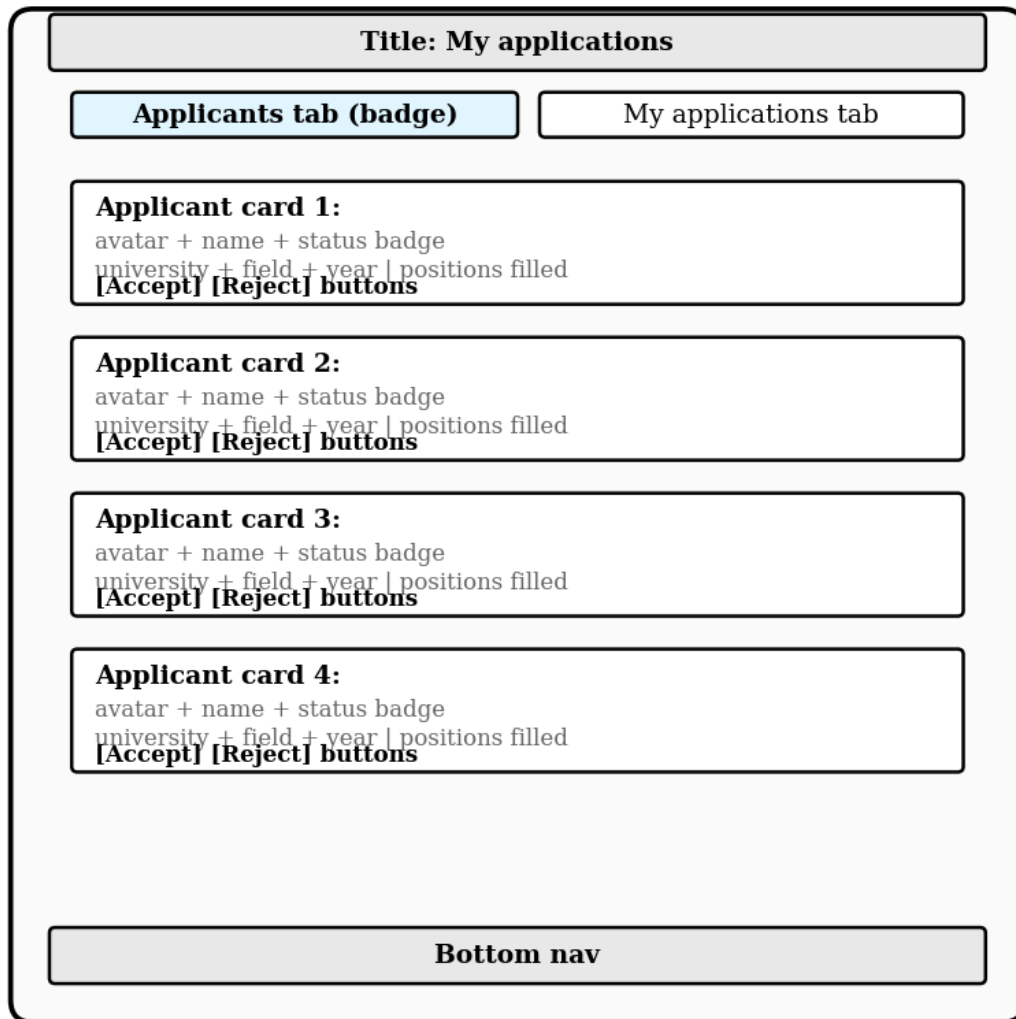


Figure 3.13: UI Diagram — Applications View

Figure 3.14: UI Diagram — Profile View

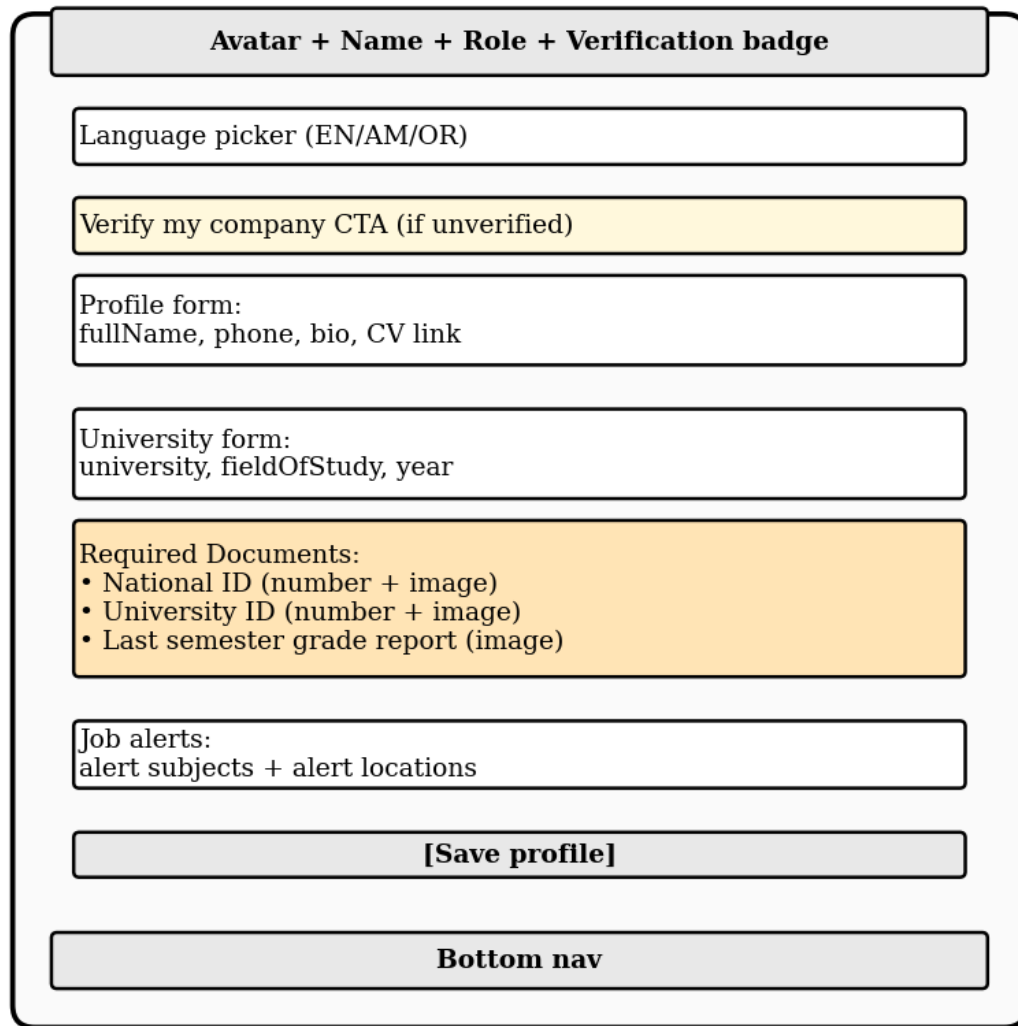


Figure 3.14: UI Diagram — Profile View

Figure 3.15: UI Diagram — Admin View

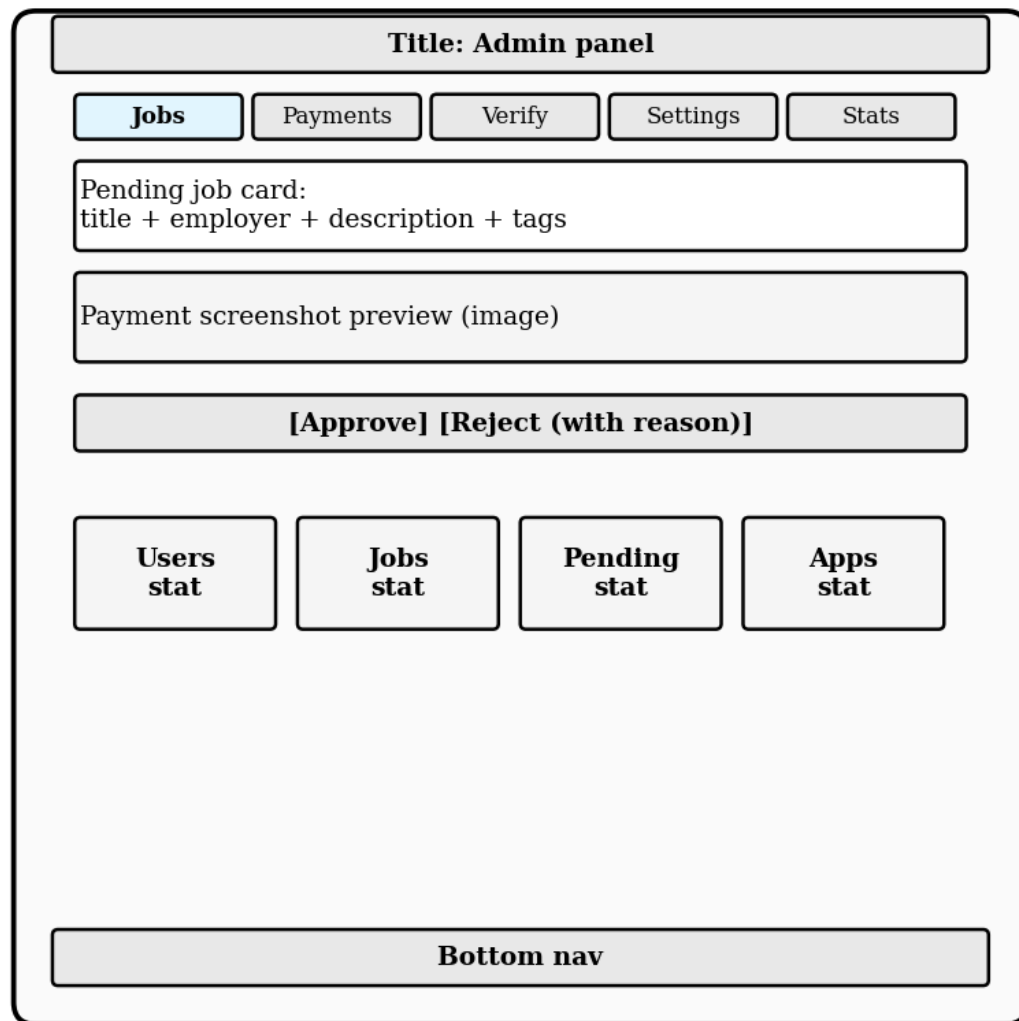


Figure 3.15: UI Diagram — Admin View

REFERENCES

- [1] Telegram. (2024). Telegram Bot API Documentation. [Online]. Available: <https://core.telegram.org/bots/api> [Accessed: July 10, 2026].
- [2] Telegram. (2024). Telegram Mini Apps — Web Apps. [Online]. Available: <https://core.telegram.org/bots/webapps> [Accessed: July 10, 2026].
- [3] Telegram. (2024). Validating Data Received via the Mini App. [Online]. Available: <https://core.telegram.org/bots/webapps#validating-data-received-via-the-mini-app> [Accessed: July 10, 2026].
- [4] Vercel Inc. (2024). Next.js 16 Documentation. [Online]. Available: <https://nextjs.org/docs> [Accessed: July 10, 2026].
- [5] Vercel Inc. (2024). Vercel Platform Documentation. [Online]. Available: <https://vercel.com/docs> [Accessed: July 10, 2026].
- [6] Prisma. (2024). Prisma ORM Documentation. [Online]. Available: <https://www.prisma.io/docs> [Accessed: July 10, 2026].
- [7] Neon. (2024). Neon Serverless Postgres Documentation. [Online]. Available: <https://neon.tech/docs> [Accessed: July 10, 2026].
- [8] Tailwind Labs. (2024). Tailwind CSS Documentation. [Online]. Available: <https://tailwindcss.com/docs> [Accessed: July 10, 2026].
- [9] shadcn. (2024). shadcn/ui Component Library. [Online]. Available: <https://ui.shadcn.com> [Accessed: July 10, 2026].
- [10] Meta Platforms. (2024). React 19 Documentation. [Online]. Available: <https://react.dev> [Accessed: July 10, 2026].
- [11] Microsoft. (2024). TypeScript Handbook. [Online]. Available: <https://www.typescriptlang.org/docs/handbook> [Accessed: July 10, 2026].
- [12] Bun. (2024). Bun JavaScript Runtime Documentation. [Online]. Available: <https://bun.sh/docs> [Accessed: July 10, 2026].
- [13] Larsson, F. and Vesterlund, M. (2023). Serverless Web Applications with Next.js and Vercel. In Proceedings of the Nordic Web Conference, pp. 45–58, Stockholm, Sweden, 2023.
- [14] Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill Education, New York, 2014.
- [15] Sommerville, I. (2015). Software Engineering (10th ed.). Pearson, Boston, 2015.

- [16] Booch, G., Rumbaugh, J., and Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley Professional, Boston, 2005.
- [17] Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, Boston, 2002.
- [18] Hoffer, J. A., Venkataraman, R., and Topi, H. (2016). *Modern Database Management* (12th ed.). Pearson, Boston, 2016.
- [19] Ethiopian Ministry of Innovation and Technology. (2022). *National Digital Transformation Strategy*. [Online]. Available: <https://www.mint.gov.et> [Accessed: July 10, 2026].
- [20] World Bank. (2023). *Ethiopia Economic Update: The Role of Digital Technology in Youth Employment*. [Online]. Available: <https://www.worldbank.org/en/country/ethiopia> [Accessed: July 10, 2026].
- [21] Afriwork. (2024). *Afriwork Freelance Ethiopia — Telegram Channel*. [Online]. Available: <https://t.me/freelanceethbot> [Accessed: July 10, 2026].
- [22] Yegna Tutors. (2024). *Yegna Tutors Harrar — Telegram Channel*. [Online]. Available: <https://t.me/Yegnatutorharrar> [Accessed: July 10, 2026].
- [23] Chapa Financial Technologies. (2024). *Chapa Payment API Documentation*. [Online]. Available: <https://developer.chapa.co> [Accessed: July 10, 2026].
- [24] Ethio Telecom. (2024). *Telebirr Mobile Payment Service*. [Online]. Available: <https://www.ethiotelecom.et/telebirr> [Accessed: July 10, 2026].
- [25] Mozilla Developer Network. (2024). *Web Crypto API — HMAC SubtleCrypto*. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto/sign> [Accessed: July 10, 2026].

APPENDICES

Appendix A: Complete CRC Card Set

This appendix contains the complete set of Class-Responsibility-Collaboration (CRC) cards for all seven primary classes in the Temariware system: User, Job, Application, Subscription, VerificationRequest, Setting, and Payment. Three representative cards were presented in Table 2.3, 2.4, and 2.5 of Chapter Two; the remaining four cards are presented here.

[Additional CRC cards for Subscription, VerificationRequest, Setting, and Payment classes — to be inserted by the team following the same format as Tables 2.3-2.5.]

Appendix B: User Interface Prototype Screenshots

This appendix contains the user interface diagrams of the six primary views of the Temariware Mini App. The diagrams show the structural layout and component composition of each view, including the header, navigation, content areas, and action buttons. These wireframes were produced during the design phase and informed the high-fidelity implementation in React with Tailwind CSS and shadcn/ui components.

Figure 3.10: UI Diagram — Home View

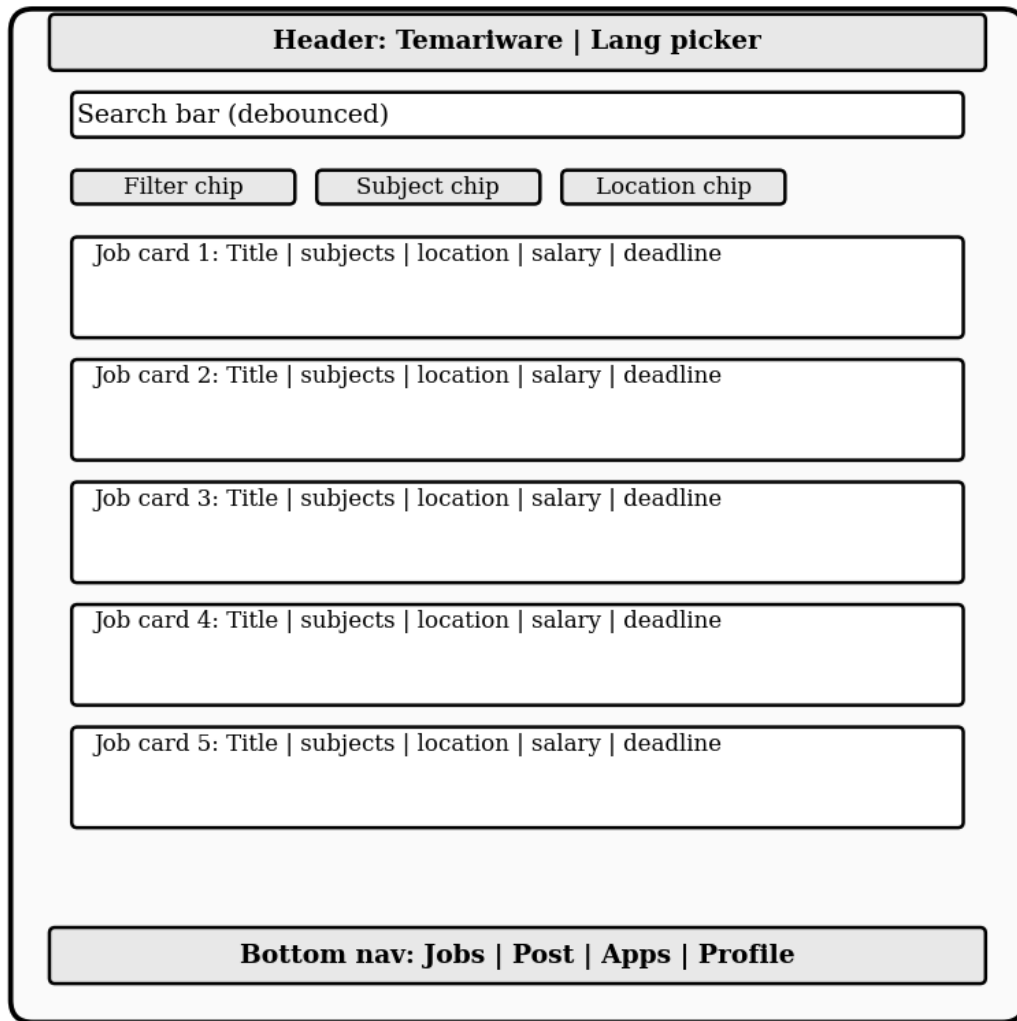


Figure B.1: Home View — Job Feed with Search and Filters

Figure 3.11: UI Diagram — Job Detail View

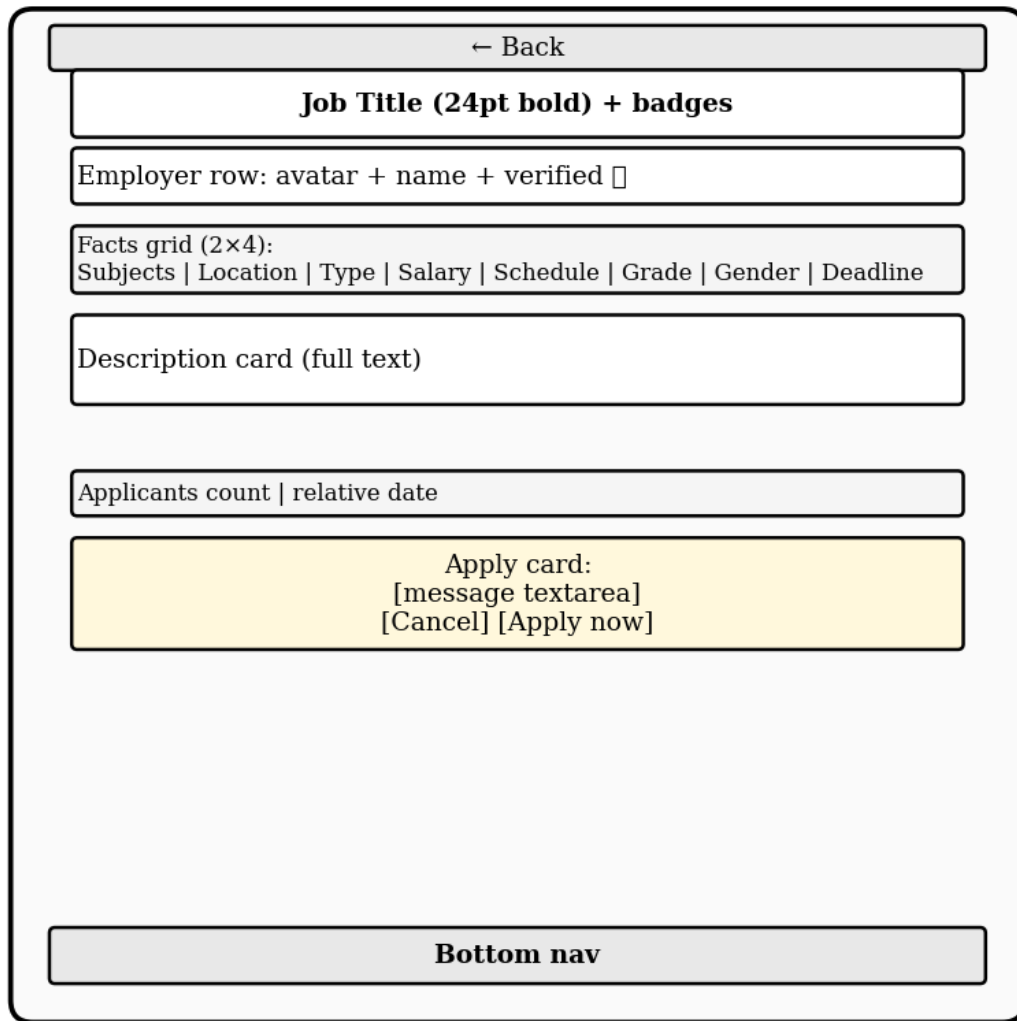


Figure B.2: Job Detail View with Apply Form

Figure 3.12: UI Diagram — Post Job View

The diagram illustrates the layout of the 'Post Job View' as a vertical stack of components within a rounded rectangle. At the top is a grey header bar labeled 'Employer badge | Title'. Below this is a light blue bar for 'Fee notice: 500 ETB + payment instructions'. The form continues with four white input fields: 'Form: title, description, subjects, grade', 'Form: location, workType, salary, schedule', 'Form: gender, language, deadline, contact', and 'Positions available (number input)'. This is followed by an 'Application fee (per-job, ETB)' field. A yellow bar contains the 'Payment screenshot uploader' with subtext '[file picker → preview → remove]'. Below this is a grey bar for the '[Submit for review button]'. At the bottom is a grey bar for the 'Bottom nav'.

Employer badge | Title

Fee notice: 500 ETB + payment instructions

Form: title, description, subjects, grade

Form: location, workType, salary, schedule

Form: gender, language, deadline, contact

Positions available (number input)

Application fee (per-job, ETB)

Payment screenshot uploader
[file picker → preview → remove]

[Submit for review button]

Bottom nav

Figure B.3: Post Job Form with Payment Screenshot Upload

Figure 3.13: UI Diagram — Applications View

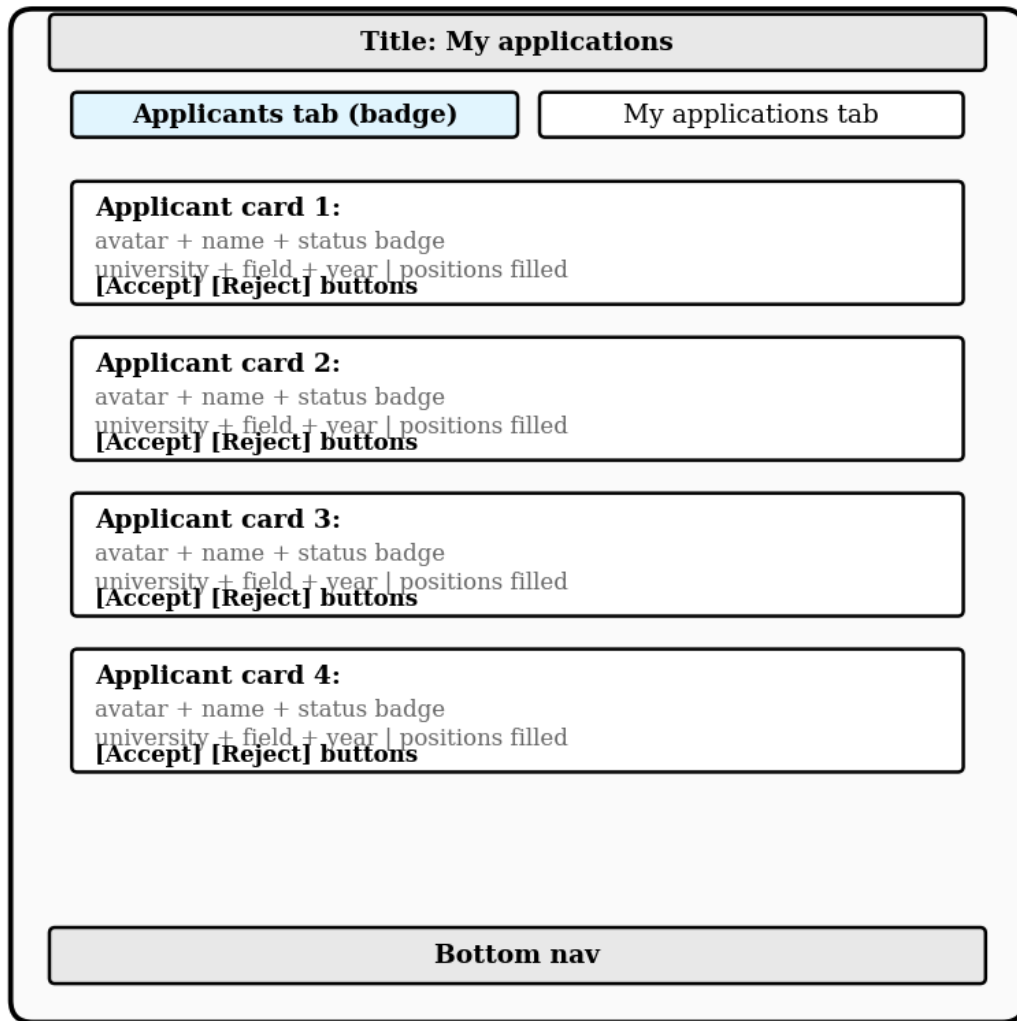


Figure B.4: Applications View — Employer Applicants Tab

Figure 3.14: UI Diagram — Profile View

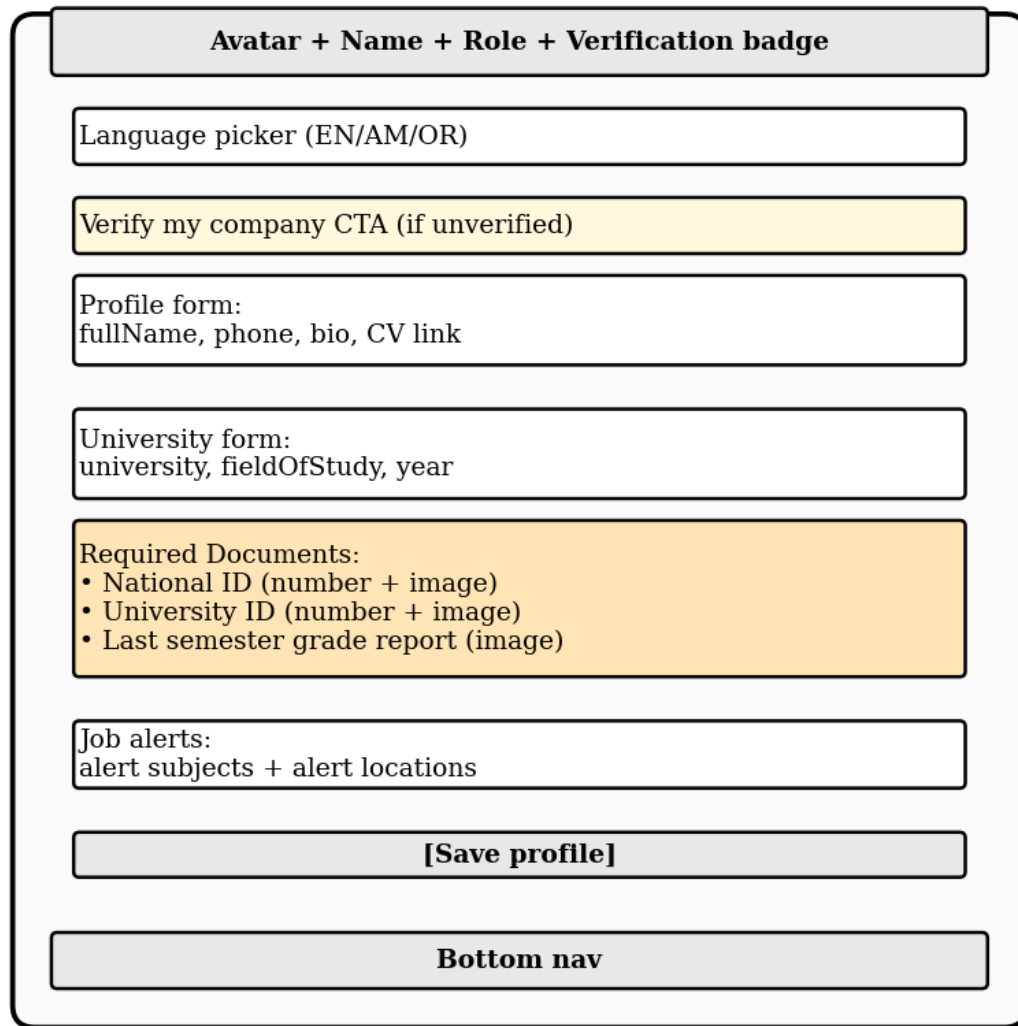


Figure B.5: Profile View — Mandatory Documents Upload

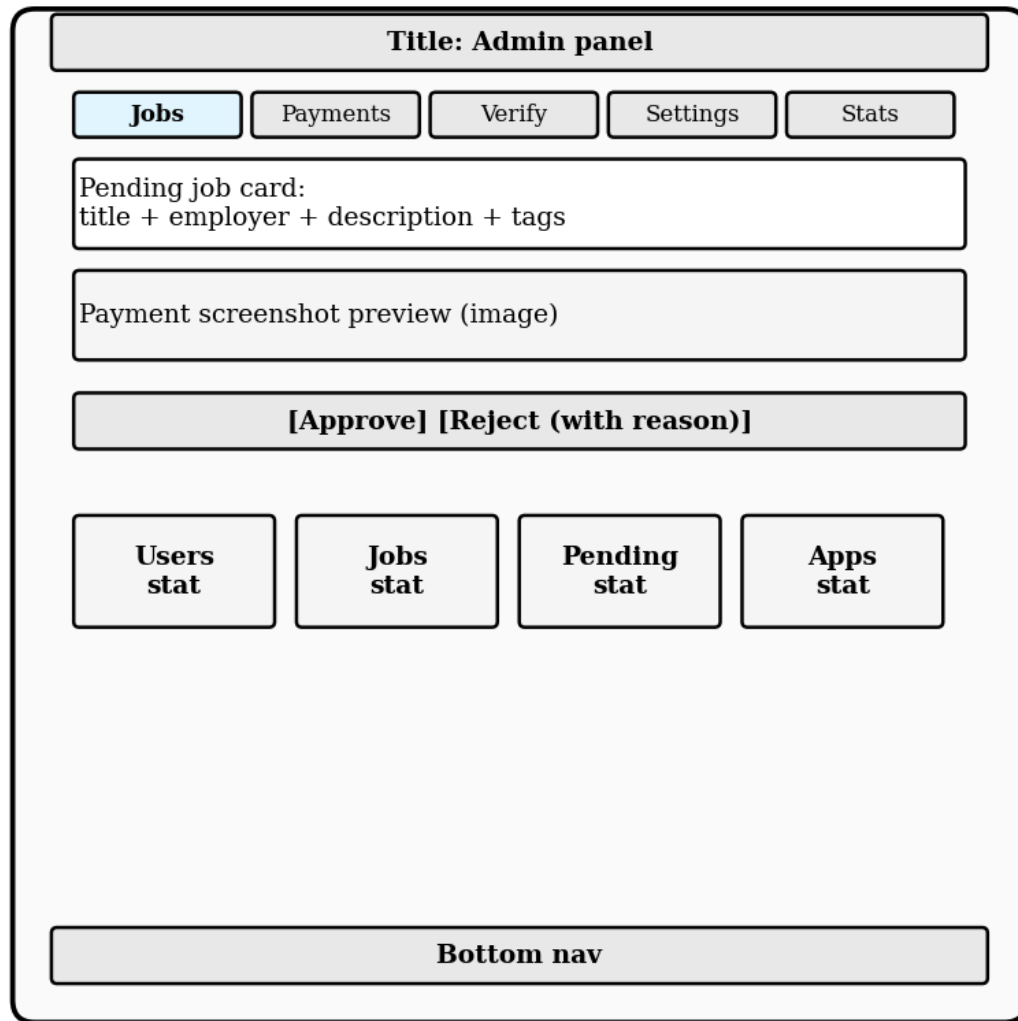
Figure 3.15: UI Diagram — Admin View

Figure B.6: Admin Panel — Pending Jobs Tab

Appendix C: Database Schema (Prisma)

This appendix contains the complete Prisma schema file (schema.prisma) that defines the database structure for Temariware. The schema is the authoritative source for all entity definitions, relationships, and constraints referenced throughout this document.

```
generator client {  provider = "prisma-client-js" }
datasource db {  provider = "postgresql"  url      = env("DATABASE_URL") }
enum UserRole { STUDENT EMPLOYER ADMIN }
enum VerificationStatus { NONE PENDING APPROVED REJECTED }
enum JobStatus { PENDING_PAYMENT PENDING APPROVED REJECTED CLOSED EXPIRED }
enum JobType { TUTORING FREELANCE PART_TIME FULL_TIME INTERNSHIP REMOTE }
enum PaymentStatus { NONE PENDING APPROVED REJECTED }
enum ApplicationStatus { PENDING ACCEPTED PAYMENT_PENDING HIRED REJECTED CLOSED }
enum PaymentType { JOB_POST APPLICATION }
model User {  id String @id @default(cuid())  telegramId String @unique  username String?  role UserRole @default(STUDENT)  language String @default("en")  fullName String?  phone String?  university String?  fieldOfStudy String?  year Int?  bio String?  cvUrl String?  nationalIdNumber String?  nationalIdImage String?  universityIdNumber String?  universityIdImage String?  lastSemesterGradeImage String?  profileComplete Boolean @default(false)  companyName String?  companyTelegram String?  verification VerificationStatus @default(NONE)  isAdmin Boolean
```

```
@default(false) alertSubjects String? alertLocations String? createdAt
DateTime @default(now()) updatedAt DateTime @updatedAt jobs Job[]
applications Application[] subscriptions Subscription[] verifications
VerificationRequest[] payments Payment[] } model Job { id String
@id @default(cuid()) title String description String employerId String
employer User @relation(fields: [employerId], references: [id]) subjects
String gradeLevel String? location String? workType JobType
@default(TUTORING) salary String? currency String @default("ETB")
hoursPerWeek String? genderPref String? languageReq String? deadline
DateTime? contactTelegram String? status JobStatus @default(PENDING_PAYMENT)
rejectionReason String? positionsAvailable Int? filledPositions Int @default(0)
applicationFee Float? paymentScreenshot String? paymentStatus PaymentStatus
@default(NONE) createdAt DateTime @default(now()) updatedAt DateTime
@updatedAt applications Application[] payments Payment[] @@index([status,
createdAt]) } model Application { id String @id @default(cuid())
jobId String job Job @relation(fields: [jobId], references:
[id]) userId String user User @relation(fields: [userId],
references: [id]) message String? status ApplicationStatus @default(PENDING)
paymentScreenshot String? paymentStatus PaymentStatus @default(NONE) hiredAt
DateTime? createdAt DateTime @default(now()) @@unique([jobId, userId])
@@index([userId]) } model Subscription { id String @id @default(cuid())
userId String user User @relation(fields: [userId], references: [id])
category String createdAt DateTime @default(now()) @@unique([userId, category]) }
model VerificationRequest { id String @id @default(cuid())
userId String user User @relation(fields: [userId],
references: [id]) companyName String contactTelegram String proofUrl String?
note String? status VerificationStatus @default(PENDING) createdAt
DateTime @default(now()) } model Setting { id String @id
@default("singleton") jobPostFee Float @default(500) applicationFee
Float @default(25) paymentInstructions String telebirrNumber String?
cbeAccount String? chapaEnabled Boolean @default(false) updatedAt
DateTime @updatedAt } model Payment { id String @id @default(cuid())
userId String user User @relation(fields: [userId],
references: [id]) jobId String job Job @relation(fields: [jobId], references: [id]) type
PaymentType applicationId
String? amount Float screenshotUrl String? status PaymentStatus
@default(PENDING) reviewedBy String? reviewNote String? createdAt
DateTime @default(now()) reviewedAt DateTime? @@index([status, type]) }
```

Appendix D: Telegram Bot Commands Reference

This appendix lists all Telegram bot commands supported by @temariwarebot, along with their descriptions and example usage.

- /start — Welcome message with Open Temariware button.
- /latest — Shows the 5 most recently approved jobs with inline buttons.
- /search <query> — Searches jobs by title, description, subjects, or location. Example: /search math
- /subscribe <category> — Subscribes to push alerts for a job category. Categories: ALL, TUTORING, FREELANCE, PART_TIME, FULL_TIME, INTERNSHIP, REMOTE. Example: /subscribe Tutoring
- /unsubscribe <category> — Unsubscribes from alerts for a category.
- /myapps — Lists the user's applications with current status.
- /postjob — Opens the Post Job form in the Mini App (verified employers only).
- /admin — Opens the Admin panel in the Mini App (administrators only).
- /help — Lists all available commands.

Appendix E: Deployment Guide Summary

This appendix summarises the deployment steps for the Temariware system. The full deployment guide is available in the project repository as `DEPLOYMENT.md`. The system is deployed entirely on free-tier infrastructure: Vercel for the Next.js application, Neon for the PostgreSQL database, and the Telegram Bot API (which is free). Total recurring cost: zero Ethiopian Birr per month.

- Step 1: Create a Telegram bot via `@BotFather` and obtain the bot token.
- Step 2: Create a Neon Postgres database and obtain the connection string.
- Step 3: Deploy the Next.js application to Vercel via the Vercel CLI.
- Step 4: Set environment variables on Vercel (`TELEGRAM_BOT_TOKEN`, `DATABASE_URL`, `MINI_APP_URL`, `ADMIN_TELEGRAM_IDS`, `ALERT_CHAT_ID`, `CHANNEL_USERNAME`, `TELEGRAM_BOT_USERNAME`).
- Step 5: Set the Telegram webhook to `https://your-app.vercel.app/api/bot`.
- Step 6: Configure the Mini App menu button in `@BotFather`.
- Step 7: Add `@temariwarebot` as administrator to the `@TEMARIWARE` channel with Post Messages permission.
- Step 8: Push the Prisma schema to the database (`prisma db push`) and seed initial settings.
- Step 9: Test end-to-end by posting a job as admin, approving it, and verifying the channel broadcast.